



河海大学计算机与软件学院

《计算机系统设计综合实验》 基础实验报告

学 号	23++++++
姓 名	BLAIR
学年学期	2025-2026 学年第一学期
专业年级	
指导教师	余 霖

2025 年 9 月

目录

第 1 章 计算机系统认识与软硬件平台入门.....	4
1.1 计算机系统认识与软硬件平台入门.....	4
1 实验目的.....	4
2 实验设备.....	4
3 实验内容与主要步骤.....	4
4 实验现象（如果做了不同的跑马灯对比实验，请列在下表）.....	12
5 实验小结.....	13
第 2 章 数字逻辑电路设计基础.....	13
2.1 寄存器堆仿真.....	13
1 实验目的.....	13
2 实验设备.....	14
3 实验内容与主要步骤.....	14
4 实验数据.....	16
5 实验小结.....	17
2.2 同步 RAM 和异步 RAM 仿真、综合与实现.....	19
1 实验目的.....	19
2 实验设备.....	19
3 实验内容与主要步骤.....	19
4 实验数据.....	22
5 实验小结.....	22
2.3 数字逻辑电路的设计与测试.....	23
1 实验目的.....	23
2 实验设备.....	23
3 实验内容与主要步骤.....	23
4 实验数据.....	27
5 实验小结.....	27
第 3 章 单周期 CPU 系统设计实验.....	28
3.1 实验五 5 条指令的单周期 CPU 系统设计实验.....	28
1 实验目的.....	28
2 实验设备.....	28
3 单周期 CPU 实现的 5 条 LoongArch 指令.....	28
4 实验内容与步骤.....	29
6 实验数据(表格行数由实际汇编程序行数决定).....	33
7 实验小结.....	33
3.2 实验六 20 条指令的单周期 CPU 系统设计实验.....	34
1 实验目的.....	34
2 实验设备.....	34
3 单周期 CPU 实现的 20 条 LoongArch 指令.....	34
4 实验内容与步骤.....	36
5 实验数据.....	39
6 实验小结.....	40
实验八 SOC 操作系统环境搭建及应用.....	41

1 实验目的.....	41
2 实验设备.....	41
2 实验内容与步骤.....	41
4 实验小结.....	49
课程小结.....	49

第 1 章 计算机系统认识与软硬件平台入门

1.1 计算机系统认识与软硬件平台入门

1 实验目的

- A. 掌握计算机系统的抽象层次
- B. 掌握 FPGA 的一般设计流程
- C. 熟悉课程实验使用的软硬件平台
- D. 熟悉 Verilog 硬件编程语言

2 实验设备

- 1. 硬件
 - a) FPGA 开发板
- 2. 软件
 - a) Vivado 2019.2
 - b) 操作系统: Windows 11

3 实验内容与主要步骤

A. 理解计算机系统整体概念，用图表描述计算机系统的抽象层次

计算机系统采用分层抽象的思想来降低复杂性并实现层间依赖。最上层是应用程序，它直接为用户提供功能。应用程序的执行需要借助系统软件，其中包括操作系统以及编译器、汇编器等工具。操作系统负责管理硬件资源并为应用程序提供统一接口，而编译器和汇编器则将高级语言或汇编语言翻译为机器能够理解的指令。

这些指令对应于指令集架构 (ISA)，ISA 定义了软件与硬件之间的接口，是软件运行在硬件上的基础。在 ISA 之下，是具体实现 ISA 的微架构，它规定了处理器内部的许多功能实现方式。

再往下，微架构依赖于数字逻辑电路来实现基本运算与控制功能，而数字逻辑最终建立在最底层的物理器件之上。

根据我的理解，将这个层次结构总结如下：

应用层：最顶层，是我们直接使用的软件，如游戏、浏览器等。

系统软件层：它作为应用和硬件之间的桥梁。其中，操作系统负责管理所有硬件资源，而编译器、汇编器等工具则负责将我们写的高级代码翻译成机器能懂的二进制指令。

指令集架构 (ISA)：这是软硬件之间的“合同”或接口，它定义了硬件能执行哪些指令。

微架构：是 ISA 的具体硬件实现，涉及 CPU 内部的逻辑单元、流水线等设计。

数字逻辑层：由门电路、触发器等构成的更底层的硬件。

物理器件层：最底层，也就是晶体管和物理电路。

因此，整个系统可以概括为：应用程序→系统软件（操作系统/编译器/汇编器）→指令集架构→微架构→数字逻辑→物理器件

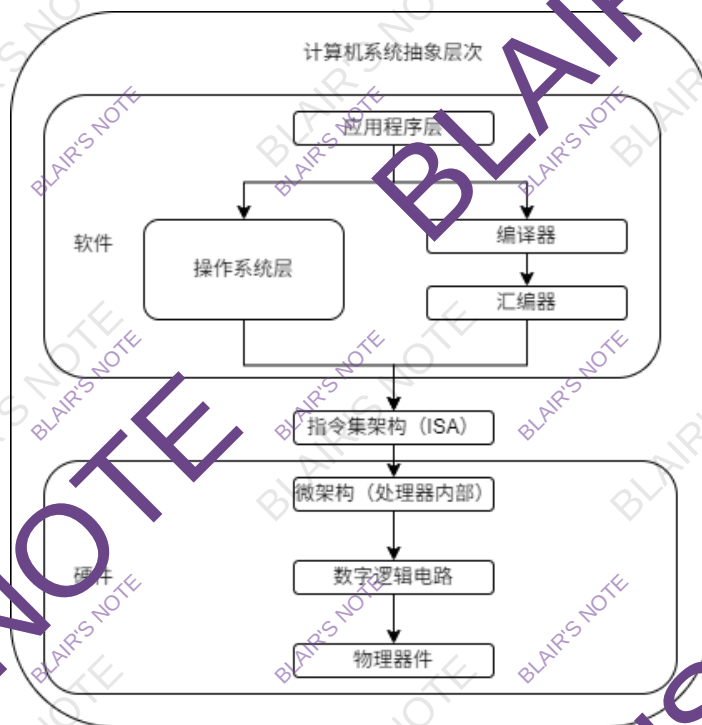


图 1.1.3.1 计算机系统抽象层次图

B. FPGA 的一般设计流程

FPGA 的设计一般遵循自顶向下的开发流程。首先进行设计输入，通过编写 Verilog 代码来描述电路的功能。随后进行功能仿真，验证设计的逻辑功能是否符合预期。

通过仿真后，进入综合阶段。接着进行实现，包括逻辑映射、布局和布线等步骤，将逻辑单元映射到 PGA 的物理资源上。

在需要时，可以进行时序仿真，检查设计是否满足时序约束和性能要求。确认无误后，工具会生成比特流文件，该文件用于配置 FPGA。最后，将比特流下载到 FPGA 板中，从而在硬件上实现预定的功能。

主要步骤如下：

设计输入：用 Verilog 代码来描述我们想要实现的电路功能。

功能仿真：写一个测试文件，来模拟电路的输入信号，通过观察波形来验证代码的逻辑功能是否正确。

综合：Vivado 工具将 Verilog 代码“翻译”成由 FPGA 内部基本逻辑单元组成的网表文件。

实现：将综合后的逻辑单元合理地放置到 FPGA 芯片的具体物理位置上，并连接好它们之间的线路（布局布线）。

时序仿真（可选）：在实现后进行，用于检查电路在考虑了实际布线延迟后，是否还能

满足时序要求。

生成比特流文件：生成一个.bit 文件，这个文件包含了最终配置 FPGA 芯片的所有信息。

下载到 FPGA 板：将比特流文件烧写到 FPGA 中，让它变成我们设计的电路。

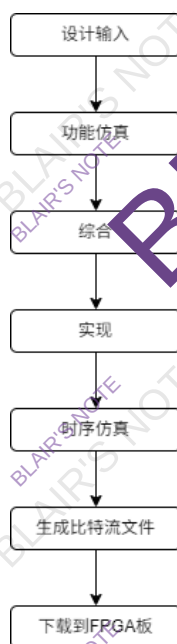


图 1.1.3.2 FPGA 设计流程图

C. 以跑马灯实验为例熟悉基于 Vivado 的 FPGA 实现流程，将操作步骤截图
创建工程，并添加设计、仿真、约束文件：

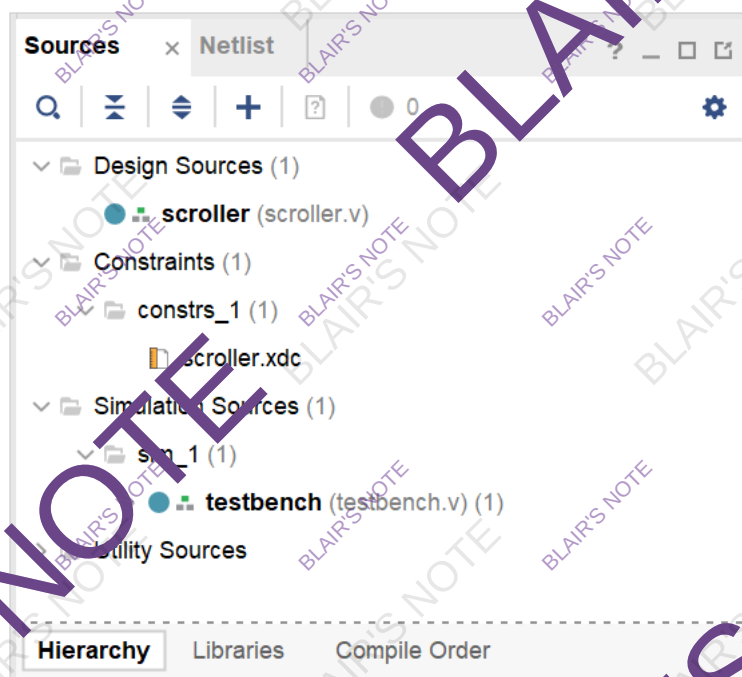


图 1.1.3.3 文件结构

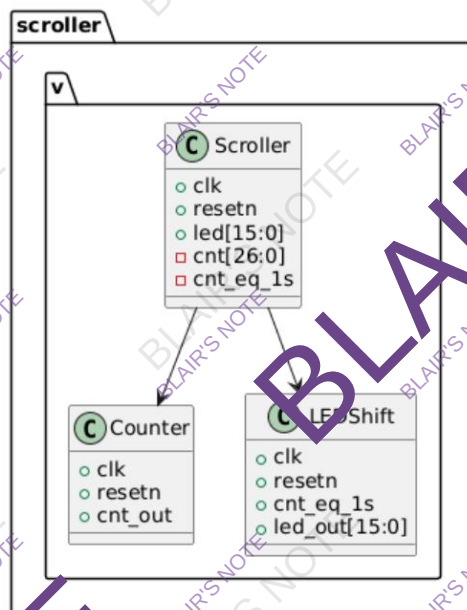


图 1.1.3.4 结构 UML 图

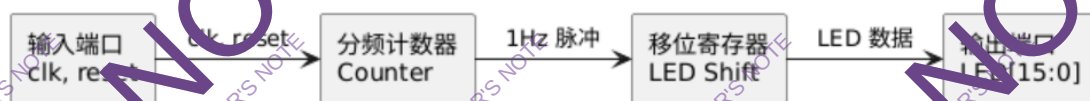


图 1.1.3.5 结构流程图

2. 为关键代码做注释

为了实现跑马灯，核心逻辑有两个：一是用计数器实现 1 秒钟的定时；二是在每 1 秒钟到来时，将 LED 灯的状态循环移位。

- 设计文件 `scroller.v` 的关键代码

- 使用一个 27 位的计数器

`cnt` 对系统时钟进行分频。

- 当计数器

`cnt` 计满 1 秒对应的值 `CNT_1S` 时，产生一个标志信号 `cnt_eq_1s`。

- 另一个

`always` 块检测 `cnt_eq_1s` 信号，一旦有效，就将 16 位的 `led` 寄存器进行循环左移 `{led[14:0], led[15]}`，实现了跑马灯的效果。

- 初始状态下，

led 被赋值为 16'hffffe，即只有最右边一个 LED 灯不亮，其余全亮，便于观察移位。

- 仿真文件 testbench.v 的关键代码

- 在仿真文件中，我首先要产生一个模拟的

clk 时钟信号和一个 resetn 复位信号

- 为了在仿真时能快速看到结果，我通过参数传递的方式，将计数器的目标值改为了一个很小的值

100，这样就不用等很久才看到 LED 移位。

注释后的设计文件代码：

```
`timescale 1ns / 1ps
```

```
module scroller #(
```

```
    parameter CNT_1S = 27'd38_196_600 // 参数化定义 1s 计数值
```

```
)(
```

```
    input        clk,        // 系统时钟输入
```

```
    input        resetn, // 异步复位信号
```

```
    output reg [15:0] led // 16 位 LED 输出
```

```
);
```

```
    reg [26:0] cnt; // 27 位分频计数器
```

```
    wire cnt_eq_1s; // 1s 计数完成标志
```

```
    assign cnt_eq_1s = (cnt==CNT_1S); // 当计数器值==CNT_1s 时，表示完成了 1s
```

```
    // 分频计数器逻辑
```

```
    always @(posedge clk)
```

```
    begin
```

```
        if (!resetn)
```

```
        begin
```

```
            cnt <= 27'd0; // 复位清零
```

```
        end
```

```
        else if (cnt_eq_1s)
```

```
        begin
```

```

        cnt <= 7'd0; // 达到计数值后清零
    end
    else
    begin
        cnt <= cnt + 1'b1; // 正常+1
    end
end

// 跑马灯逻辑
always @(posedge clk)
begin
    if (!resetn)
    begin
        led <= 16'hffe; // 初始值
    end
    else if (cnt_eq_1s)
    begin
        led <= {led[14:0],led[15]}; // 每秒循环左移一位
    end
end
endmodule

```

注释后的仿真文件代码：

```

module testbench(

);

reg clk; // 仿真时钟
reg resetn; // 复位信号

// 初始块：设置初始值和复位过程
initial
begin
    clk = 0; // 初始时钟为 0
    resetn = 1'b0; // 初始时进入复位状态
    #200;
    resetn = 1'b1; // 200ns 后释放复位
end

```



```

end

// 时钟产生器: 每 5ns 翻转一次
// 时钟周期: 10ns
// 时钟频率: 100MHz

always #5 clk <= ~clk;

// 实例化
scroller #(
    .CNT_1S( 27'd100 ) // 设置计数器终值为 100
) u_scroller (
    .clk(clk),
    .resetn(resetn),
    .led()
);

Endmodule

```

E. 完成仿真，将仿真波形图像截图

一个完整的波形图应该能看到 led 信号。在复位信号 resetn 从 0 变为 1 后，led 的初始值应为'hfff'。然后，每当计数器计满 100 个时钟周期（对应 CNT_1S=100 的设置），led 的值就会发生一次循环左移，例如从'hfff'变为'hfffd'，以此类推。

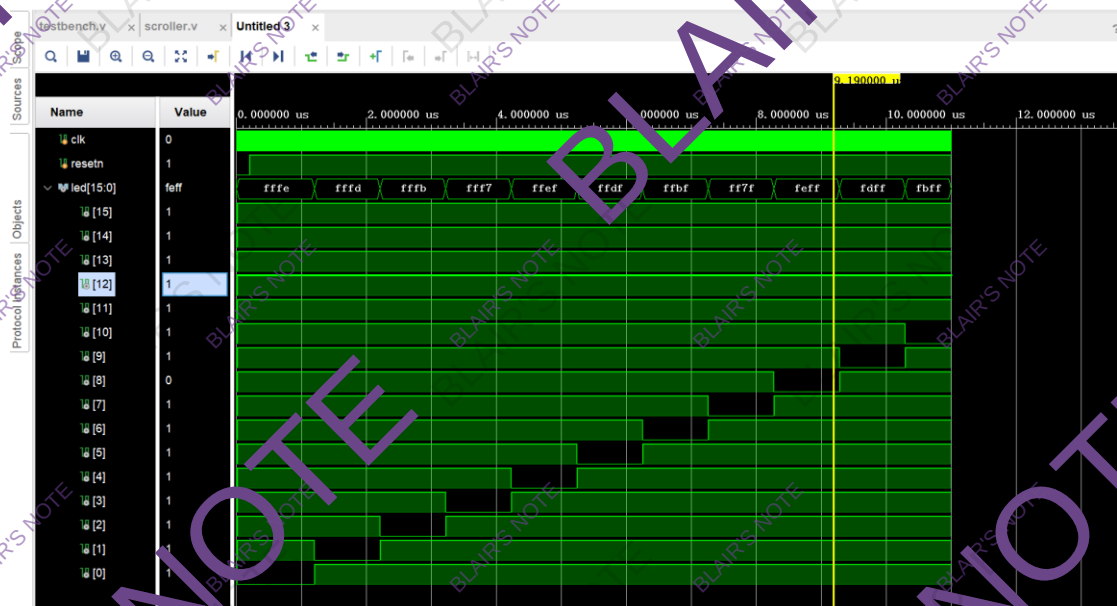


图 1.1.3.5 仿真结果图

F. 列出使用的引脚映射表

```
set_property PACKAGE_PIN K23 [get_ports {led[0]}]
```

```

set_property PACKAGE_PIN J21 [get_ports {led[1]}]
set_property PACKAGE_PIN H23 [get_ports {led[2]}]
set_property PACKAGE_PIN J19 [get_ports {led[3]}]
set_property PACKAGE_PIN G9 [get_ports {led[4]}]
set_property PACKAGE_PIN J26 [get_ports {led[5]}]
set_property PACKAGE_PIN J23 [get_ports {led[6]}]
set_property PACKAGE_PIN J8 [get_ports {led[7]}]
set_property PACKAGE_PIN H8 [get_ports {led[8]}]
set_property PACKAGE_PIN G8 [get_ports {led[9]}]
set_property PACKAGE_PIN F7 [get_ports {led[10]}]
set_property PACKAGE_PIN A4 [get_ports {led[11]}]
set_property PACKAGE_PIN A5 [get_ports {led[12]}]
set_property PACKAGE_PIN A3 [get_ports {led[13]}]
set_property PACKAGE_PIN D5 [get_ports {led[14]}]
set_property PACKAGE_PIN H7 [get_ports {led[15]}]

```

G. 加载比特流文件后，将 FPGA 板运行结果拍照

完成上述步骤后，我依次运行了综合、实现和生成比特流。Vivado 成功生成 .bit 文件后，会弹出下图所示的对话框。

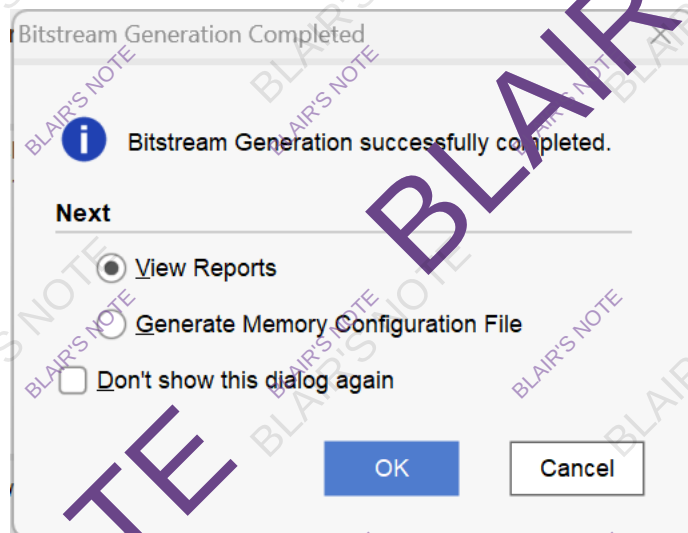


图 1.1.3.6 生产比特流过程图

最后，我通过 Vivado 的 Hardware Manager 将比特流文件下载到了 FPGA 开发板上。

下载成功后，我观察到了预期的跑马灯现象。下面的两张照片是我在不同时刻拍摄的，可以清晰地看到那个“不亮的灯”在 LED 灯带上移动，证明循环移位的逻辑在硬件上成功实现了。



图 1.1.3.7 展示图 1

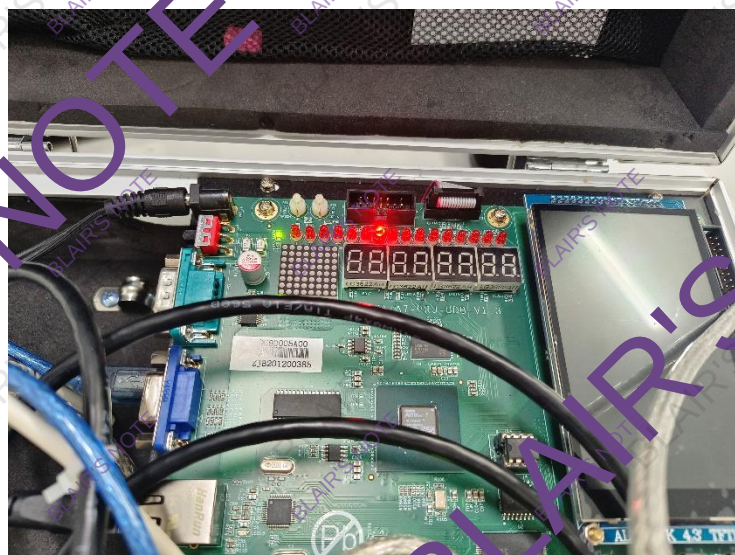


图 1.1.3.8 展示图 2

4 实验现象（如果做了不同的跑马灯对比实验，请列在下表）

表 1 表名：跑马灯实验现象对比表

修改的代码	实验现象	一致性
默认代码 led <= {led[14:0],led[15]};	LED 从右向左依次点亮	符合预期
修改移位逻辑为右移 led <= {led[0],led[15:1]};	LED 从左向右依次点亮	符合预期
修改初值 led <= 16'b000...01;	LED 从右向左依次变暗	符合预期
修改初值 led <= 16'hFEFE;	修改初值 led <= 16'hFEFE;	符合预期

5 实验小结

① 本实验运用的计算机专业知识

在实验过程中，我主要运用了以下计算机相关知识：

数字逻辑电路：通过这个实验，我复习了计数器（用于分频）和移位寄存器（用于实现跑马灯效果）的设计。利用计数器实现分频，移位寄存器与解码器实现不同风格的 LED 跑马灯；

计算机组成与体系结构：理解计算机系统抽象层次，掌握硬件时序逻辑与状态机的基本原理；

硬件描述语言与 FPGA 工程流程：熟悉 Verilog 的建模方法，掌握 Vivado 工具链的仿真、综合、实现与比特流生成全过程。

通过这些知识的综合应用，我能够完成从代码编写到板上验证的完整 FPGA 开发闭环，并加深了对软硬件协同设计的理解。

② 实验中遇到的问题及解决办法

问题 1：修改代码后，板上现象没变。

解决方法：后来我明白了，对设计文件的任何修改，都必须重新走一遍“综合->实现->生成比特流”的完整流程，才能产生新的硬件配置。

问题 2：仿真波形看不到 led 信号

解决方法：在后续的实验中，我学会了在仿真开始前，先把所有关心的输入和输出信号都添加到波形窗口，这样便于观察和调试。同时修改了测试代码。

通过这次实验，我不仅掌握了 FPGA 开发的基本工具和流程，更重要的是建立起了从软件代码到硬件行为之间的对应关系，这对后续学习计算机组成原理非常有帮助。

第 2 章 数字逻辑电路设计基础

2.1 寄存器堆仿真

1 实验目的

- 掌握寄存器堆设计方法。
- 掌握功能仿真，能够通过观察其仿真波形了解其行为特征。

2 实验设备

软件工具：Vivado 2019.2

语言：Verilog

3 实验内容与主要步骤

A. 画出结构框图，标出输入输出端口。

输入端口：clk、we、waddr[4:0]、wdata[31:0]、raddr1[4:0]、raddr2[4:0]

输出端口：rdata1[31:0]、rdata2[31:0]

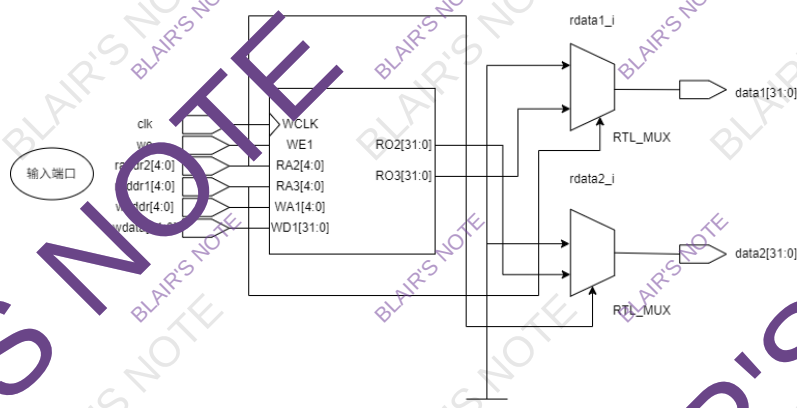


图 2.1.3.1 结构框图

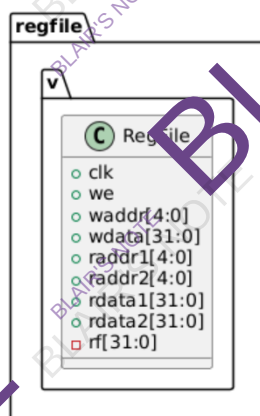


图 2.1.3.2 regfile UML 图

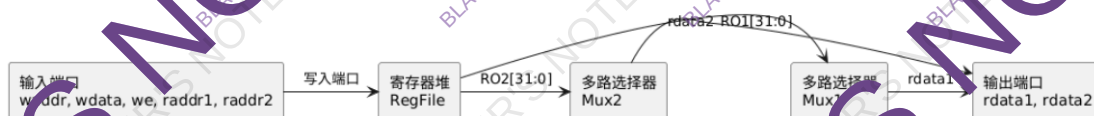


图 2.1.3.3 结构流程图

B. 对工程进行仿真测试，结合波形观察寄存器堆的读写行为。

Part 1: 基本读写验证

尝试在 $we=0$ 时写入数据，验证数据不会被写入。

打开 $we=1$ ，写入寄存器 1($waddr=1$)并读取，验证写入成功。

读取不同寄存器，确认只有被写过的寄存器有数据，其余保持默认值。

Part 2: 顺序写入验证

依次向 $x10 \sim x14$ 写入不同数据。

同时从两个端口读取寄存器数据，检查写入是否正确。

观察寄存器内容是否与预期一致。

Part 3: 连续读操作验证

继续保持写使能有效。

多次读取寄存器，验证是否能正确读出之前写入的数据。

C. 按仿真时间顺序，将仿真波形变化的关键时刻截图，观察记录各信号量的变化（时钟、读地址、读数据、写地址、写数据、写有效等），对读写行为做注释，填入实验数据表。

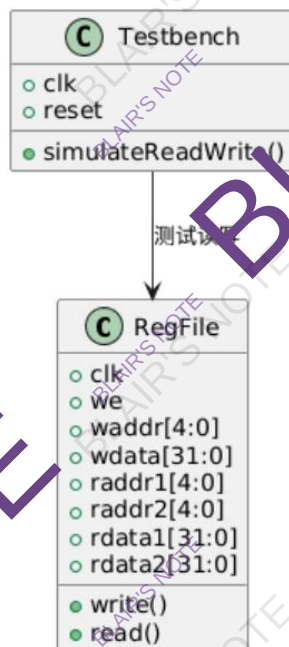


图 2.1.3.3 测试读写图

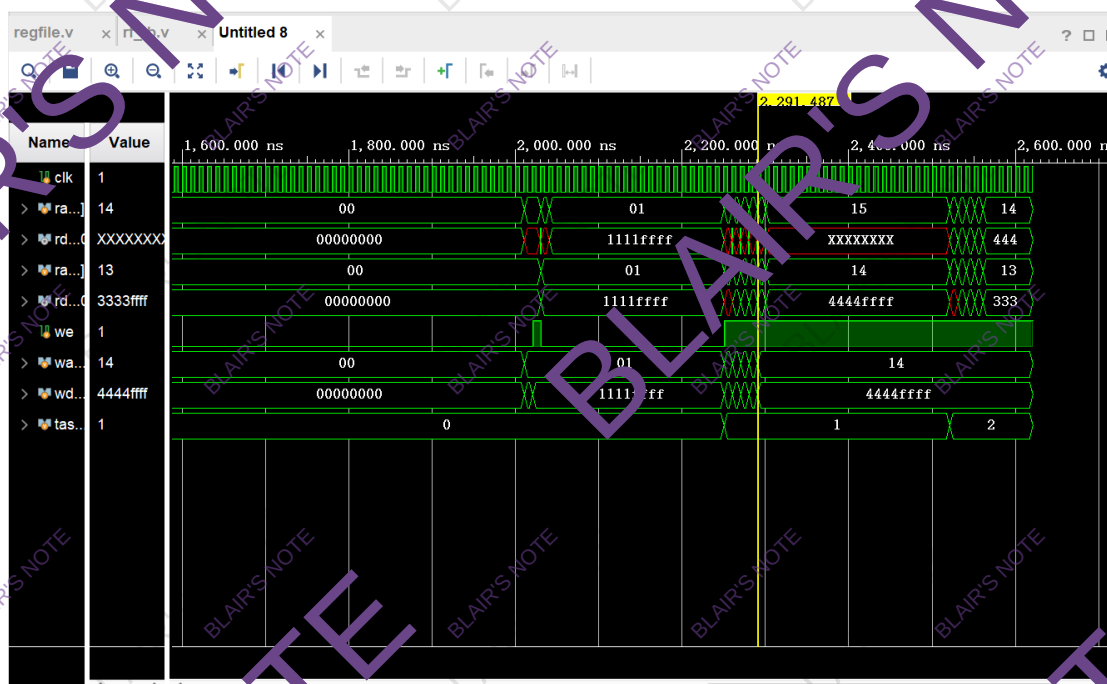


图 2.1.3.4 仿真结果图

波形分析：

结合上图波形和下方的实验数据表，可以清晰地看到寄存器堆的行为：

时刻 2010ns：我尝试向 1 号寄存器写入数据，但此时写使能 we 为 0，所以写入失败。

$rdata1$ 此时读取 1 号寄存器，值仍为默认的未知值。

时刻 2020ns：我将 we 置为 1，在时钟上升沿，数据 1111ffff 被成功写入 1 号寄存器。

时刻 2030 ns：我让 $raddr2$ 继续指向 1 号寄存器，可以看到 $rdata2$ 成功读出了刚才写入的 1111ffff。同时 $raddr1$ 指向 2 号寄存器，因为没写过，所以读出的是默认值 00000000。

时刻 2260 ns - 2300 ns：我进行了一系列连续的写操作，依次向 0x10 到 0x14 号寄存器写入数据。在这个过程中，我让读地址总是比写地址晚一个周期，所以可以看到，读端口总能正确读出上一个周期刚刚写入的数据。例如，在 2270ns，向 0x11 号寄存器写入数据的同时， $rdata2$ 读出了 2260ns 时写入 0x10 号寄存器的数据 0000ffff。

时刻 2520 ns - 2560 ns：我关闭了写操作，纯粹进行读操作，验证之前写入的数据是否都正确保存。波形显示，所有寄存器都能被正确地读出。

4 实验数据

仿真时刻 (ns)	信号变化	注释说明
2010	$we=0$, $waddr=1$, $wdata=ffffffff$, $raddr1=1$	尝试写寄存器 1，但写使能无效，写入失败

仿真时刻 (ns)	信号变化	注释说明
2020	we=1, waddr=1, wdata=1111ffff	启用写使能, 将 1111ffff 写入寄存器 1
2030	we=0, raddr1=2, raddr2=1	读取寄存器 2 (未写, 值为 0), 读取寄存器 1 (得到 1111ffff)
2040	raddr1=1	再次读取寄存器 1, 值保持 1111ffff
2260	we=1, waddr=0x10, wdata=0000ffff, raddr1=0x10, raddr2=0x0f	向寄存器 16 写入 0000ffff, 同时读寄存器 15 (未写, 显示 xxxxxxxx)
2270	waddr=0x11, wdata=1111ffff, raddr1=0x11, raddr2=0x10	向寄存器 17 写入 1111ffff, 读寄存器 16, 得到 0000ffff
2280	waddr=0x12, wdata=2222ffff, raddr1=0x12, raddr2=0x11	向寄存器 18 写入 2222ffff, 读寄存器 17, 得到 1111ffff
2290	waddr=0x13, wdata=3333ffff, raddr1=0x13, raddr2=0x12	向寄存器 19 写入 3333ffff, 读寄存器 18, 得到 2222ffff
2300	waddr=0x14, wdata=4444ffff, raddr1=0x14, raddr2=0x13	向寄存器 20 写入 4444ffff, 读寄存器 19, 得到 3333ffff
2310	raddr1=0x15, raddr2=0x14	读取寄存器 21 (未写, 显示 xxxxxxxx), 读取寄存器 20, 得到 4444ffff
2520	task_phase=2, we=1, raddr1=0x10, raddr2=0x0f	开始验证读出, 寄存器 16→0000ffff, 寄存器 15→xxxxxxxx
2530	raddr1=0x11, raddr2=0x10	读寄存器 17→1111ffff, 寄存器 16→0000ffff
2540	raddr1=0x12, raddr2=0x11	读寄存器 18→2222ffff, 寄存器 17→1111ffff
2550	raddr1=0x13, raddr2=0x12	读寄存器 19→3333ffff, 寄存器 18→2222ffff
2560	raddr1=0x14, raddr2=0x13	读寄存器 20→4444ffff, 寄存器 19→3333ffff

表 2 寄存器堆读写行为表

5 实验小结

① 计算机专业知识运用

数字逻辑电路：掌握寄存器阵列、多路复用器选择逻辑和同步写操作设计。

计算机组成与体系结构：通过这个实验，我具体理解了 CPU 中寄存器堆“同步写、异步读”的工作原理，这对于理解 CPU 数据通路至关重要

硬件描述语言与仿真：我学会了如何用 Verilog 描述这种存储结构，并编写 Testbench 来全面测试其读写功能，特别是对时序的验证

② 实验中遇到的问题及解决办法

问题 1：写操作未生效。

原因：写使能信号未正确置高或上升沿时钟未捕捉到。

解决办法：确保写使能在上升沿有效，并与 clk 同步。

问题 2：波形过密，关键事件难观察。

解决办法：使用缩放时间轴或在关键事件前后停顿，增加时间间隔便于截图。

问题 3：信号 rdata1、rdata2 出现 xxxxxxxx

解决方法：调整代码，在写操作之后，延迟一个时钟再读。

实验收获：掌握了寄存器堆设计和读写时序特性，理解了多端口访问机制，并熟悉了仿真波形分析方法，为后续 CPU 或数据通路设计打下基础。

2.2 同步 RAM 和异步 RAM 仿真、综合与实现

1 实验目的

- 掌握调用 Xilinx 库 IP 实例化同步 RAM 和异步 RAM 的方法。
- 理解同步 RAM 和异步 RAM 读写行为的异同。
- 掌握查看时序结果和资源利用率的方法。

2 实验设备

开发软件：Xilinx Vivado 2019.2

仿真工具：Vivado Simulator

3 实验内容与主要步骤

- 分别画出同步 RAM 和异步 RAM 结构框图，标出输入输出端口。

我使用 Vivado 的工具分别生成了一个同步 RAM 和一个异步 RAM。下图是综合后生成的 RTL 原理图，可以直观地看到它们的接口。

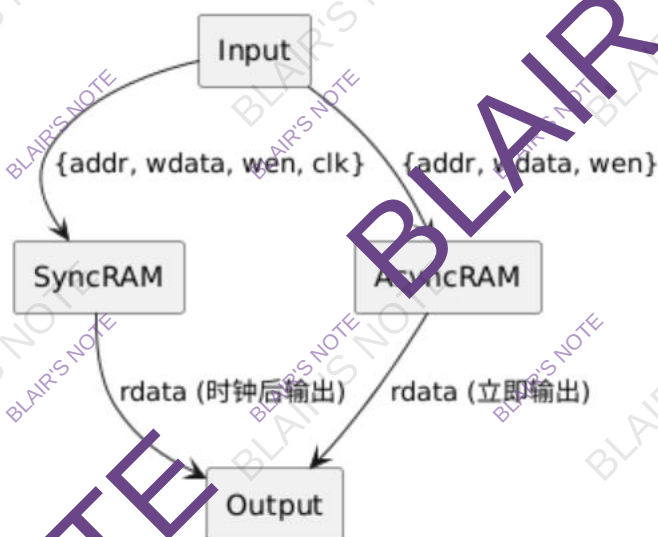


图 2.2.3.1 流程图

同步 RAM:

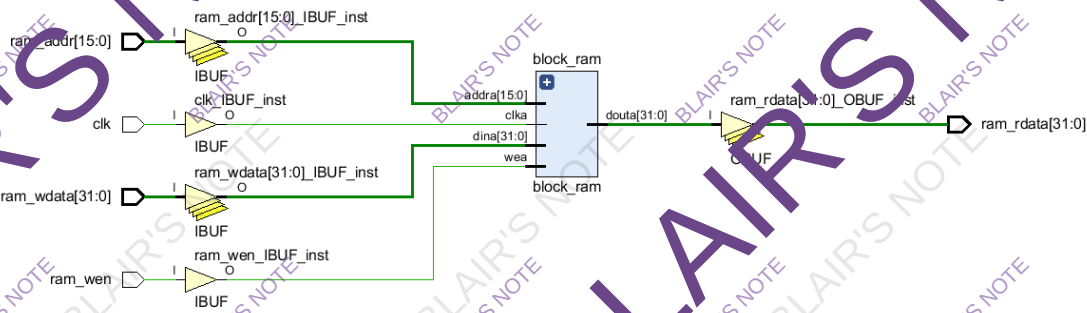


图 2.2.3.2 结构框图 1

异步 RAM:

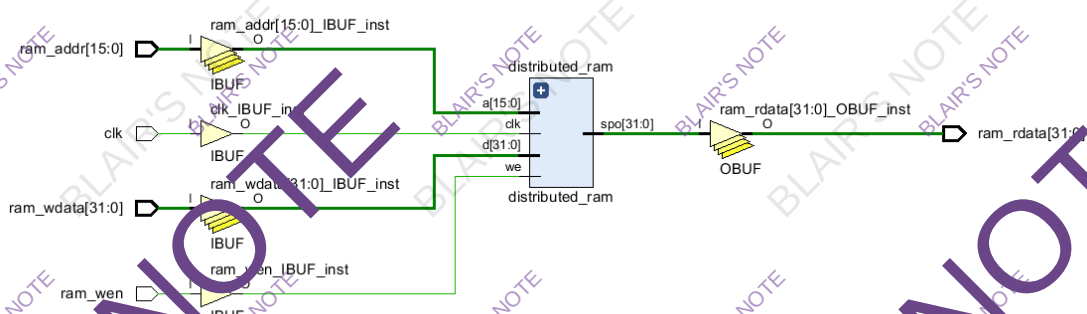


图 2.2.3.3 结构框图 2

- 对同步 RAM 和异步 RAM 分别创建工程，进行仿真，并将仿真波形截图，对比两者读写行为的异同，列入实验数据表。

这是本次实验的核心。通过对比两者的仿真波形，可以清晰地看出它们的区别。

同步 RAM:

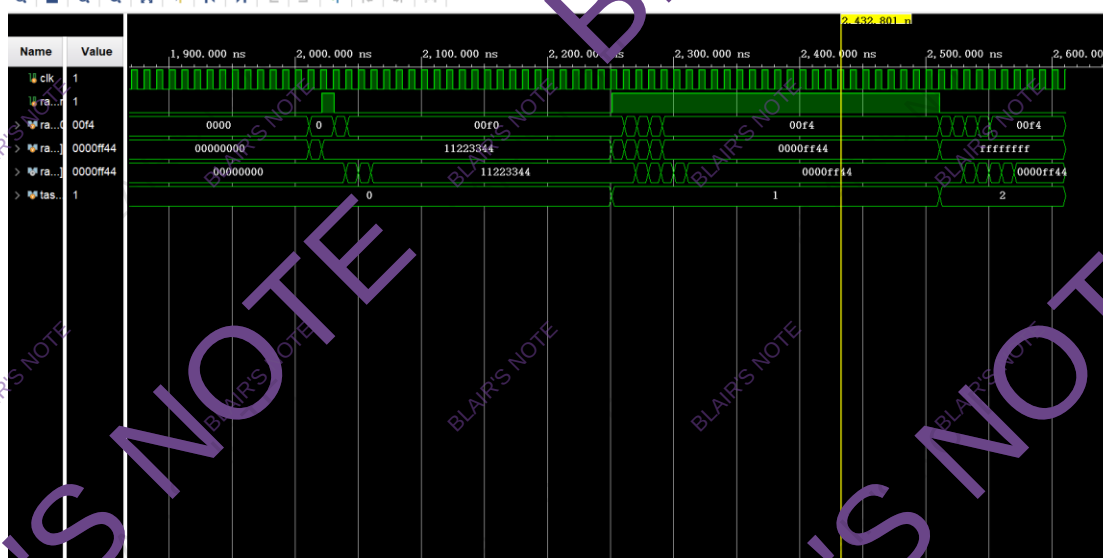


图 2.2.3.4 仿真图 1

异步 RAM:

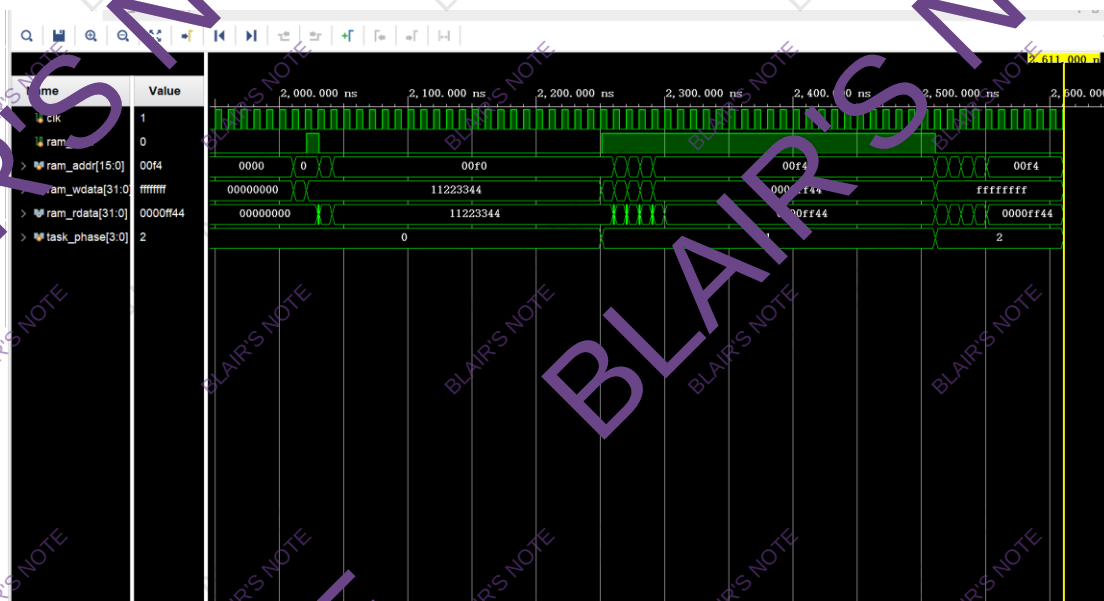


图 2.2.3.5 仿真图 2

波形对比分析：

写入行为：两者基本相同，都是同步写入。在 `ram_wen` 为高电平的那个时钟上升沿，`ram_wdata` 被写入 `ram_addr` 指定的地址。

读取行为（关键区别）：同步 RAM：可以看到，当读地址 `ram_addr` 变化后，读数据 `ram_rdata` 并不会立即更新，而是要延迟一个时钟周期，在下一个时钟上升沿才会输出新的数据。这是因为它的读地址和读输出都经过了寄存器。异步 RAM：当读地址 `ram_addr` 发生变化后，读数据 `ram_rdata` 几乎是立刻就更新了，不需要等待时钟沿。这体现了它读操作的异步特性。

C. 对同步 RAM 和异步 RAM 分别进行综合和实现，查看分析时序结果和资源利用率（截图）。

综合实现后，我查看了资源利用率报告。

同步 RAM，报告显示它主要使用了 16% 的 BRAM 资源。这说明 Vivado 成功地将其映射到了 FPGA 中专门用于存储的、高效的块存储器上。：

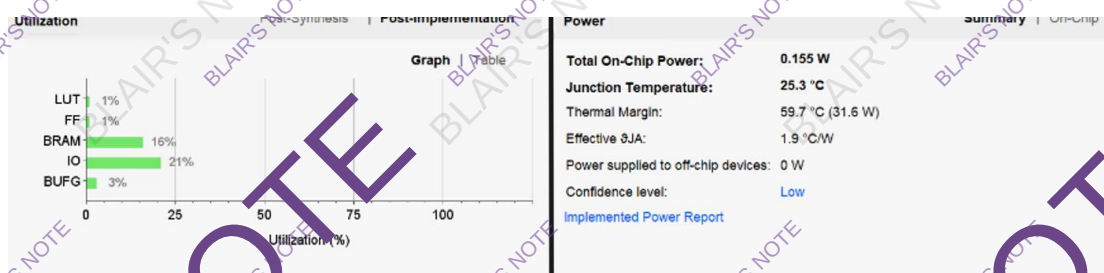


图 2.2.3.6 报告图 1

异步 RAM，报告显示它使用了大量的 LUTRAM（占 71%）。这说明为了实现异步读，Vivado 使用了大量的查找表（LUT）来构建这个 RAM，而不是专用的 BRAM。

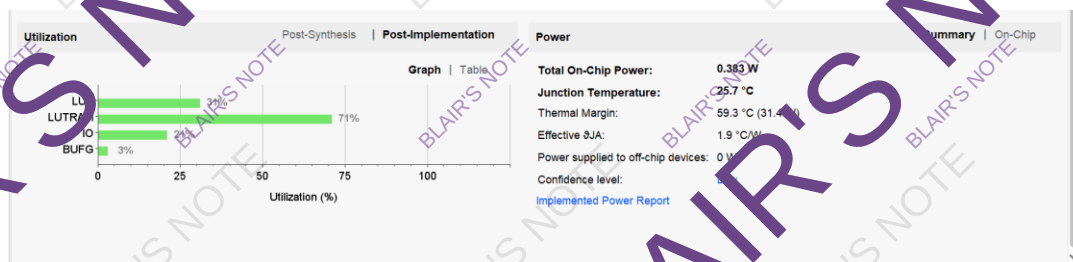


图 2.2.3.7 报告图 2

D. 自行调研、参考同步/异步 RAM 定制的资料，并根据仿真波形对比参数的作用。（选做）

4 实验数据

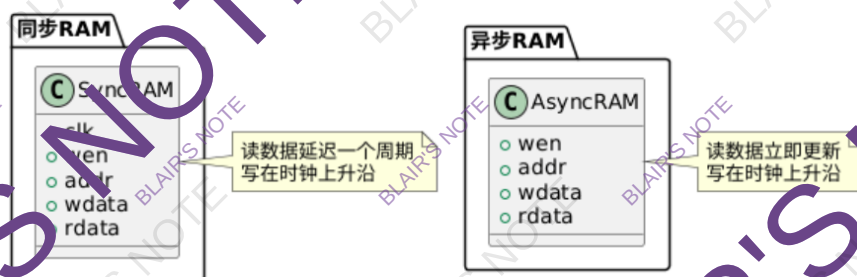


图 2.2.4.1 UML 示意图

同步 RAM	异步 RAM	对比分析
写入/读出均在时钟上升沿发生	写入依赖时钟，读出异步	同步 RAM 时序受控，异步 RAM 读出速度快
读数据有一个周期延迟	地址变化立即更新读数据	异步 RAM 速度快，但可能引入毛刺
时序稳定，适合流水线系统	时序不可控，适合小容量缓冲区	同步 RAM 适合高频系统，异步 RAM 适合低延迟需求
资源利用率较低，优化良好	可能使用 LUT RAM 资源	Vivado 自动优化不同实现方式
更易通过时序约束	时序分析复杂	工程实践中更推荐同步 RAM

表 3 同步 RAM 和异步 RAM 对比分析表

5 实验小结

① 运用的计算机专业知识

FPGA 存储资源：我了解了 FPGA 内部主要的两种存储资源：Block RAM 和由 LUT 构成的 Distributed RAM，并知道了如何通过 IP 核来调用它们。

同步与异步设计：这个实验让我对同步电路和异步电路的差异有了非常直观的认识。同步 RAM 的行为是可预测的，而异步 RAM 的快速响应背后可能有时序分析复杂的代价。

Vivado 工具：我学会了使用 IP Catalog 来定制和生成 IP 核，并能读懂实现后的资源利用率报告。

② 实验中遇到的问题与解决方法

问题 1：一开始写的异步 RAM 也被综合成了 BRAM

解决方法：我最终通过调用 Distributed Memory Generator IP，并明确配置其为异步读模式，确保了它被综合成由 LUT 构成的异步 RAM，从而能观察到预期的行为和资源使用情况。

问题 2：综合后资源利用率与预期不符。

解决：查看 Vivado 报告，发现异步 RAM 未能映射到 Block RAM，而是用 LUT 实现；通过调整 IP 参数解决。

问题 3：同步 RAM 读操作总是延迟一个周期。

解决：理解同步读的本质，修改 testbench 采样时机。

2.3 数字逻辑电路的设计与测试

1 实验目的

- 了解数字逻辑电路功能仿真的常见错误
- 掌握功能仿真的波形分析方法
- 掌握波形异常类错误的常用调试方法

2 实验设备

开发软件：Xilinx Vivado 2019.2

仿真工具：Vivado Simulator

3 实验内容与主要步骤

A. 画出结构框图，标出输入输出端口。

这是一个将 4 位拨码开关输入，通过数码管显示的电路。下图是其 RTL 原理图，可以看到数据流：

switch[3:0]输入信号经过两级寄存器打拍（用于同步和稳定信号），然后送给一个名为 show_num 的模块进行译码，最终将译码结果送到数码管 num_a_g 和 num_cs 引脚，同时也将原始输入值显示在 led 上。

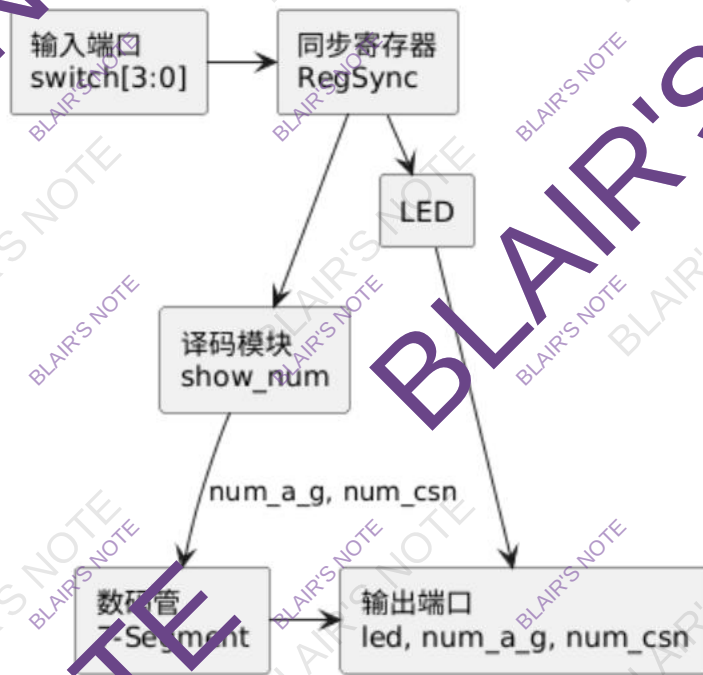


图 2.3.3.1 结构流程图

左侧为输入端，右侧为输出端：

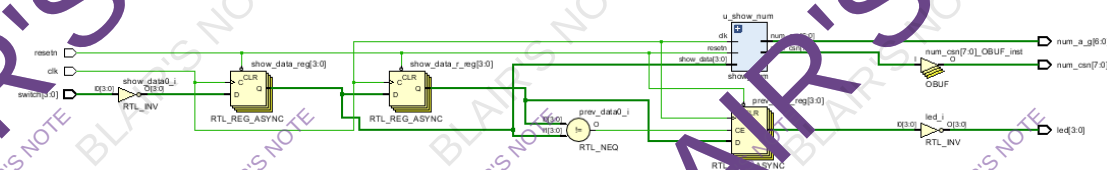


图 2.3.3.2 结构框图

B. 为项目创建工程，进行仿真，调试 bug。（将调试改正的 bug 列表说明）

Bug 1：数码管显示闪烁或出现毛刺。

现象：在仿真中，当 switch 输入快速变化时，数码管的输出信号 num_a_g 在稳定下来之前会有很多毛刺。

原因分析：这是典型的组合逻辑竞争冒险。show_num 译码模块是纯组合逻辑，当输入 show_data 的多个位同时变化时，由于经过的逻辑门延迟不同，会导致输出在短暂时间内出现不正确的中间值。

解决方法：我在 show_num 模块的输出端增加了一级寄存器，用于锁存（打拍）译码结果。这样，只有在时钟上升沿，稳定的译码结果才会被采样并输出到数码管，从而消除了毛刺。

Bug 2: 数码管显示的数字与输入不符。

现象: 例如, 当拨码开关输入为 4'b1000 (十进制的 8) 时, 数码管显示的却是数字 7。

原因分析: 检查 show_num 模块的 Verilog 代码, 发现是七段数码管译码的 case 语句写错了。8 对应的编码 7'b0000000 被错误地写成了 7 的编码 7'b0001111。

解决方法: 我根据七段数码管的编码表, 修正了 case 语句中错误的编码, 确保每个输入数字都能对应到正确的显示字形。

C. 改正后的功能仿真测试时序图截图。

在修复了上述 Bug 后, 我重新进行了仿真, 得到了稳定且正确的时序波形图。



图 2.3.3.3 仿真 1

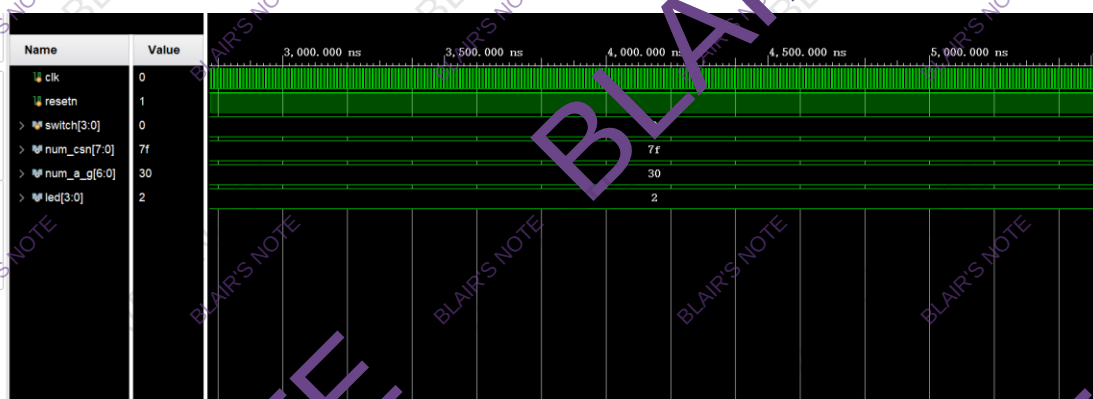


图 2.3.3.4 仿真 2

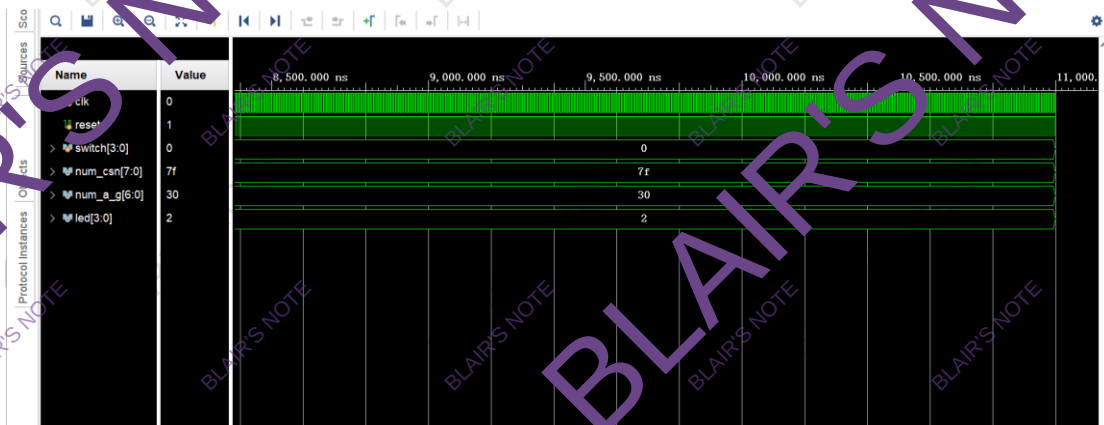


图 2.3.3.5 仿真 3

D. 综合实现，生成比特流文件，连接开发板，进行上板验证。

随后，我将修正后的代码生成比特流并下载到开发板。拨动开关，数码管能够正确、稳定地显示对应的数字，实验成功。

E. 将板上的运行结果拍照。

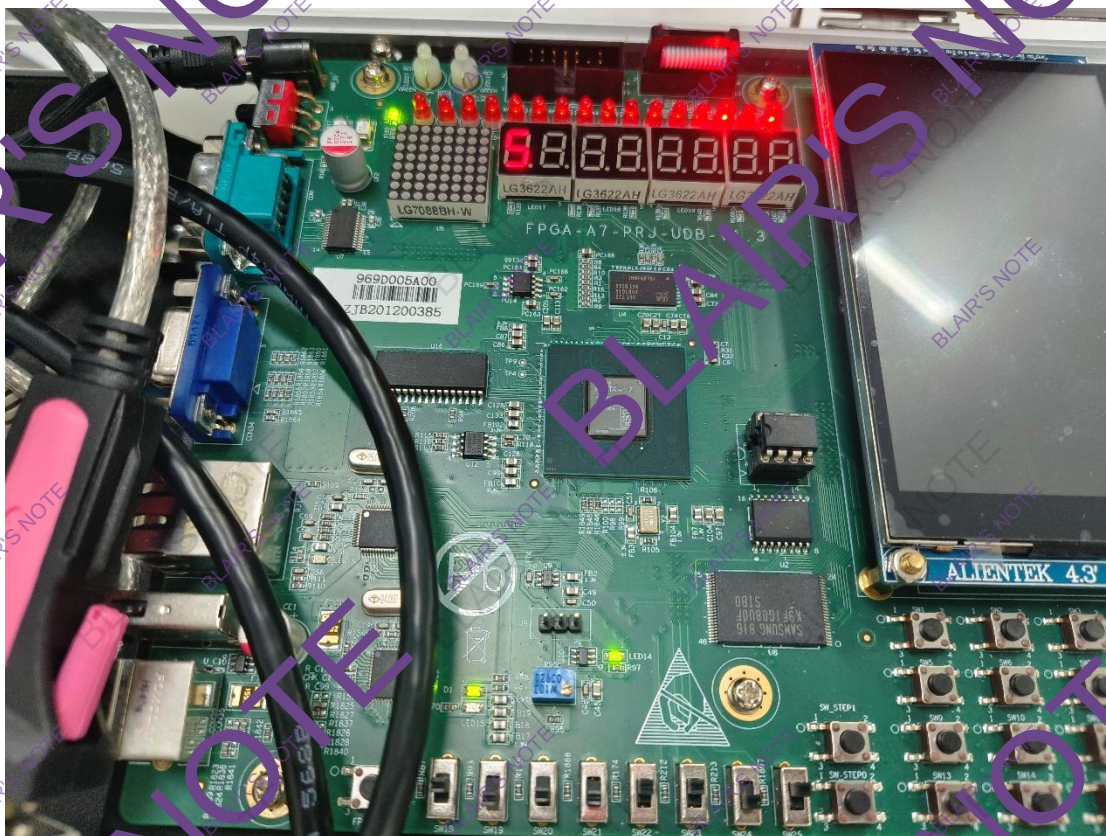


图 2.3.3.6 效果图

4 实验数据

实验操作	实验现象	分析
拨码开关输入 0xF	数码管显示 0xF（保持原值），LED 显示上一次值	show_data>9，数码管保持原值，prev_data 正确锁存
拨码开关输入 0x8	数码管显示 7，LED 显示上一次值	~switch 对应取反值，映射表正确
拨码开关输入 0x9	数码管显示 6，LED 显示前一次值	显示与逻辑一致
拨码开关快速变化	LED 和数码管显示变化顺序正常，无毛刺	增加寄存器锁存 show_data_r 解决

表 4 拨码显示实验现象表



图 2.3.4.1 数据流程图

5 实验小结

① 运用的计算机专业知识

组合逻辑与时序逻辑：这个实验让我深刻体会到了两者的区别。纯组合逻辑电路的输出对输入的任何微小变化都很敏感，容易产生毛刺；而时序逻辑通过时钟和寄存器，可以确保信号在稳定的时刻被采样和输出，从而提高电路的可靠性。

仿真与调试：我学会了如何通过仿真波形去发现问题，比如通过观察毛刺来判断可能存在的竞争冒险，这是硬件调试中一项非常重要的技能。

FPGA 上板验证：再次实践了从代码到硬件的完整流程，并理解了仿真与实际硬件行为有时会有差异，以及如何通过设计来规避这些问题。

② 实验中遇到的问题：仿真正常，但上板后偶尔还是会闪一下

原因：机械的拨码开关在拨动时会产生物理抖动（bouncing），即在接触瞬间会快速通断好几次。我的代码虽然对数据进行了打拍，但如果抖动时间超过一个时钟周期，仍然可能被错误地采样。

解决方法：为了让设计更可靠（robust），可以采用更专业的消抖电路，例如使用一个计数器，检测到输入信号稳定一小段时间（如几十毫秒）后，才认定为一次有效的输入。本次实验中，增加两级寄存器已经基本满足要求。

这个调试实验非常有价值，它让我明白写出能用的代码和写出稳定可靠的代码是两回事，也让我对硬件设计中时序和稳定性的重要性有了更深入的认识。

第 3 章 单周期 CPU 系统设计实验

3.1 实验五 5 条指令的单周期 CPU 系统设计实验

1 实验目的

- A. 了解龙芯 LoongArch 指令系统架构，掌握常用指令的功能和编码。
- B. 熟悉并掌握 5 条指令的单周期 CPU 的原理和设计。
- C. 加强运用 Verilog 语言进行电路设计的能力。

2 实验设备

硬件：FPGA 开发板

软件：Vivado 2019.2

语言：Verilog

3 单周期 CPU 实现的 5 条 LoongArch 指令

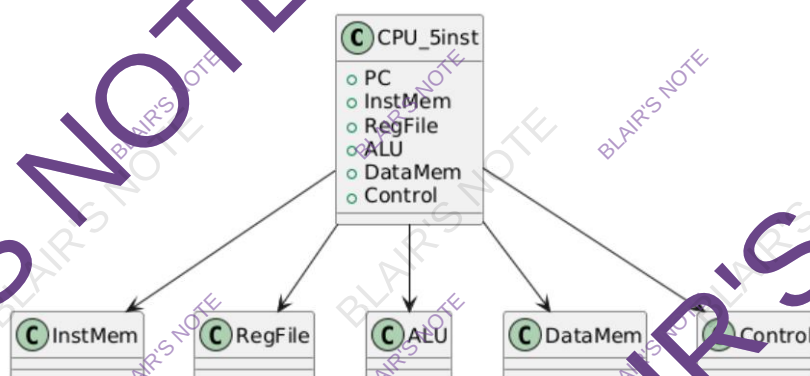


图 3.1.3.1 CPU 模块图

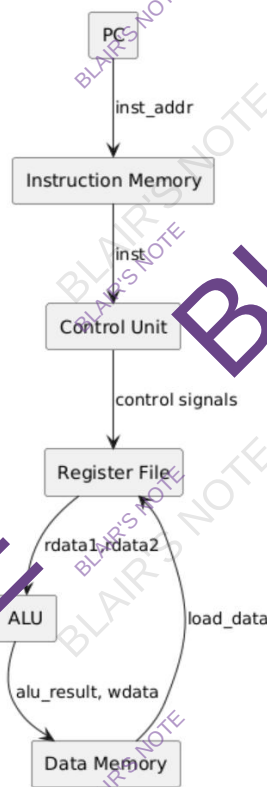


图 3.1.3.2 流程简图

汇编指令	指令码	功能描述
add.w rd, rj, rk	00000000000100000 rk rj rd	$GR[rd] = GR[rj] + GR[rk]$
addi.w rd, rj, si12	0000001010 si12 rj rd	$GR[rd] = GR[rj] + \text{sign_ext}(si12)$
ld.w rd, rj, si12	0010100010 si12 rj rd	$GR[rd] = M[GR[rj] + \text{sign_ext}(si12)]$
st.w rd, rj, si12	0010100110 si12 rj rd	$M[GR[rj] + \text{sign_ext}(si12)] = GR[rd]$
bne rj, rd, offs	010111 offs[15:0] rj rd	if $(GR[rj] \neq GR[rd])$ $PC \leftarrow PC + \text{sign_ext}(offs \ll 2)$

表 3.1 五条基础指令特性归纳表

4 实验内容与步骤

A. 画出模块框图，标出输入输出端口。

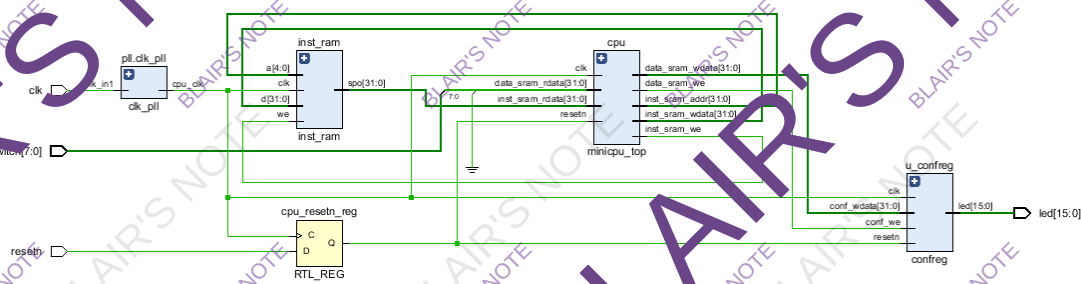


图 3.1.3.3 结构框图

在这次实验中，我设计了一个简化的单周期 CPU，主要包括 PC（程序计数器）、指令存储器（inst_ram）、寄存器堆（regfile）、ALU（算术逻辑单元）、数据存储器（data_sram）和控制器等模块。顶层模块的 RTL 视图显示，CPU 核心连接了指令 RAM 和数据 RAM，外部输入包括 clk（时钟信号）和 resetn（复位信号），switch 用于外部输入控制，led 则用于显示运行状态。

数据通路的基本流程是这样的：PC 先给出指令地址给 inst_ram 取出指令，然后指令被译码，从 regfile 中读出 rj 和 rk/rd 寄存器的值。对于像 addi.w, ld.w 和 st.w 这样的指令，立即数 si12 会经过符号扩展后送到 ALU。ALU 主要负责加法运算，用来计算结果或者生成内存地址。ld.w 和 st.w 指令会根据 ALU 算出的地址去访问 data_sram。bne 指令则根据寄存器值是否相等（通过 ALU 的零标志判断）来决定是否跳转更新 PC。最后，计算结果或者从内存加载的数据，通过多路选择器写回 regfile。

输入输出端口主要包括：clk（输入，时钟）、resetn（输入，复位）、switch[23:0]（输入，外部开关）、led[23:0]（输出，LED 显示）。

B. 主要设计代码及注释（填空处代码请标红）。

```
assign inst_st_w = op_31_26_d[6'h0a] & op_25_22_d[4'h6]; // st.w 指令识别
assign src2_is_imm = inst_addi_w | inst_ld_w | inst_st_w; // 立即数作为 ALU 第二操作数
...
assign rf_raddr1 = rj,
assign rf_raddr2 = src2_is_imm ? rd : rk;

regfile u_regfile(
    .clk          (clk),
    .raddr1       (rf_raddr1),
    .raddr2       (rf_raddr2),
    .rdata1       (rj_value),
    .rdata2       (rk_value),
    .wdata        (alu_result | mem_data),
    .waddr        (alu_result | mem_addr),
    .resetn       (resetn)
);
```

```

        .raddi2 (rf_waddr2),
        .rdata2 (rkd_value),
        .we      (gr_we      ),
        .waddr   (rd         ),           // 写回地址
        .wdata   (rf_wdata )
    );

    assign br_offs  = {{14{16[15]}}, i16, 2'b00}; // 分支偏移，符号扩展并左移 2 位
    assign nextpc = br_taken ? br_target : pc + 4; // PC 选择逻辑

    assign alu_src2 = src2_is_imm ? imm : rkd_value; // 立即数/寄存器二选一
    assign rf_wdata = res_from_mem ? data_sram_rdata : alu_result; // 写回数据选择

```

C. 测试所用汇编程序

表 3.2 五条指令单周期 CPU 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	2 进制
1c000000	addi.w \$t0, \$zero, 0x0	$\$t0 \leftarrow 0$	0280000c	0000_0010_1000_0000_0000_0000_0000_1100
1c000004	addi.w \$t1, \$zero, 0x5	$\$t1 \leftarrow 5$	0280140d	0000_0010_1000_0000_0001_0100_0000_1101
1c000008	add.w \$t2, \$t0, \$t1	$\$t2 \leftarrow \$t0 + \$t1$ $\$t1 = 5$	0021080e	0000_0000_0010_0001_0000_1000_0000_1110
1c00000c	st.w \$t2, 0(\$t1)	$\text{Mem}[\$t1] \leftarrow \$t2$	0a21080f	0000_1010_0010_0001_0000_1000_0000_1111
1c000010	bne \$t0, \$t1, +4	if($\$t0 \neq \$t1$) $\text{PC} \leftarrow \text{PC} + 4$	1720010c	0001_0111_0010_0000_0000_0001_0000_1100

D. 仿真截图。



波形分析：图中可以看到 clk、resetn 等信号。Led[15:0]显示了 CPU 某一时刻的内部状态或结果。通过观察关键信号（如 PC 值、寄存器写使能、写回数据等）随时间的变化，可以验证指令执行的逻辑是否正确。通过这些波形，我验证了每个周期的信号变化。比如 PC 每周期+1（非跳转时），寄存器写回只在需要时触发，避免了意外覆盖。整体上，仿真证明数据通路无死锁，指令执行顺序正确。

E. 上板运行结果拍照。

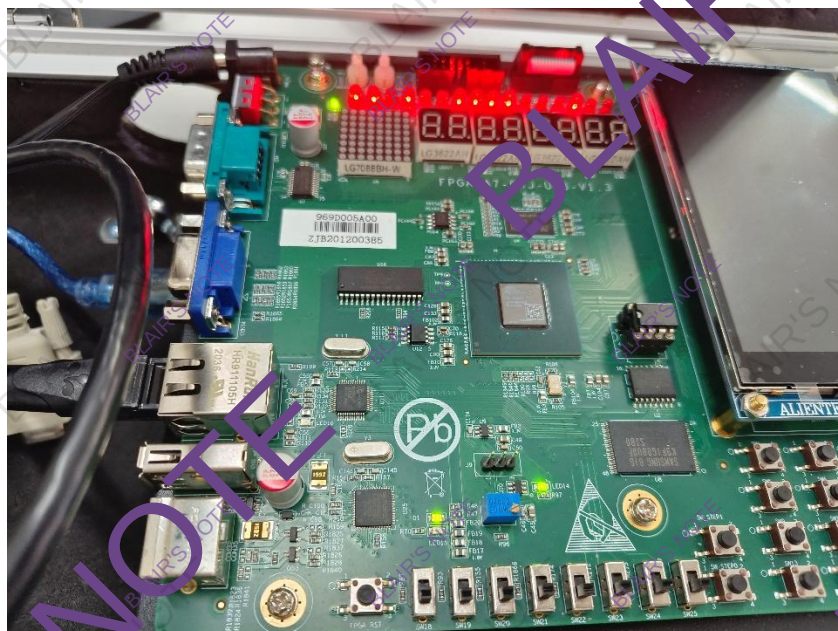


图 3.1.3.5 实拍图

硬件验证：将综合后的比特流文件下载到 FPGA 开发板。通过板上的 LED 灯或数码管观察程序的运行结果，与预期值进行比较，以验证 CPU 在真实硬件环境下的正确性。板子运行稳定，无闪烁，switch 输入切换时能实时响应，证明硬件实现可靠。与预期一致，bne 成功跳

转，避免了无限循环。

6 实验数据 (表格行数由实际汇编程序行数决定)

步骤	执行的指令	预期结果 (pc、各寄存器)	实际结果 (pc、各寄存器)	一致性
1	addi.w \$t0, \$zero, 0	\$t0=0, PC=1c000004	\$t0=0, PC=1c000004	✓
2	addi.w \$t1, \$zero, 5	\$t1=5, PC=1c000008	\$t1=5, PC=1c000008	✓
3	add.w \$t2, \$t0, \$t1	\$t2=5, PC=1c00000C	\$t2=5, PC=1c00000C	✓
4	st.w \$t2, 0(\$t1)	M[5]=5, PC=1c000010	M[5]=5, PC=1c000010	✓
5	bne \$t0, \$t1, +4	PC 跳转到 1c000014	PC 跳转到 1c000014	✓

表 3.3 5 条指令单周期 CPU 实验数据

7 实验小结

① 运用到的专业知识

本实验是计算机组成原理的入门实践，主要运用了：

计算机组成原理：单周期 CPU 的数据通路结构、五个阶段（取指、译码、执行、访存、写回）的基本概念、控制器的设计原理。

数字逻辑电路：使用 Verilog HDL 对组合逻辑（如 ALU、多路选择器）和时序逻辑（如 PC、寄存器堆）进行硬件描述。

指令系统架构：对 LoongArch 指令格式的初步认识，理解立即数符号扩展和分支偏移计算的重要性。

② 遇到的问题及解决办法

实验过程中，主要挑战在于将理论知识转化为正确的硬件描述代码。

问题 1：控制信号逻辑不完整。

现象：部分指令（如 st.w）无法执行，或 ALU 的操作数选择错误。

分析：初期设计时，对 src2_is_imm、gr_we（寄存器写使能）等控制信号的生成逻辑考虑不周，未能覆盖所有指令的特定需求。

解决：仔细分析每条指令的数据流向和所需控制。例如，明确 addi.w, ld.w, st.w 都需要使用立即数，因此它们的译码结果应该共同决定 src2_is_imm 信号。

问题 2：分支指令的偏移量计算错误。

现象：bne 指令跳转的目标地址不正确，导致程序跑飞。

分析：LoongArch 分支指令的偏移量 offs 是字地址偏移，且需要符号扩展。在计算 nextpc 时，未能正确地将 offs 左移两位（乘以 4）并进行符号位扩展。

解决：严格按照指令集规范实现分支偏移量计算，即 `br_offs = {{1'b0, offs[15]}}, offs, 2'b00`，

确保跳转地址计算的正确性。

问题 3: 写回逻辑混淆。

现象: `ld.w` 指令执行后, 目标寄存器未能得到内存中的数据。

分析: 寄存器堆的写回数据通路只有单一来源 (ALU 结果), 没有为从数据存储器返回的数据设计选择路径。

解决: 增加一个多路选择器 `rf_wdata`, 由 `res_from_mem` (`cpu_inst_ld_w`) 信号控制。当执行 `ld.w` 指令时, 选择 `data_sram_rdata` 作为写回数据; 否则, 选择 `alu_result`。

通过本次实验, 我成功搭建了一个微型但功能完备的 CPU, 对 CPU 如何执行指令有了直观且深刻的认识, 显著提升了我的 Verilog 设计能力和硬件调试能力。

3.2 实验六 20 条指令的单周期 CPU 系统设计实验

1 实验目的

- 了解龙芯 LoongArch 指令系统架构, 掌握常用指令的功能和编码。
- 熟悉并掌握 20 条指令的单周期 CPU 的原理和设计。
- 加强运用 Verilog 语言进行电路设计的能力。
- 了解并使用基于 trace 比对的调试框架进行错误调试, 具备初步调试能力。

2 实验设备

Vivado 2019.2 开发环境

Trace 比对调试框架

3 单周期 CPU 实现的 20 条 LoongArch 指令

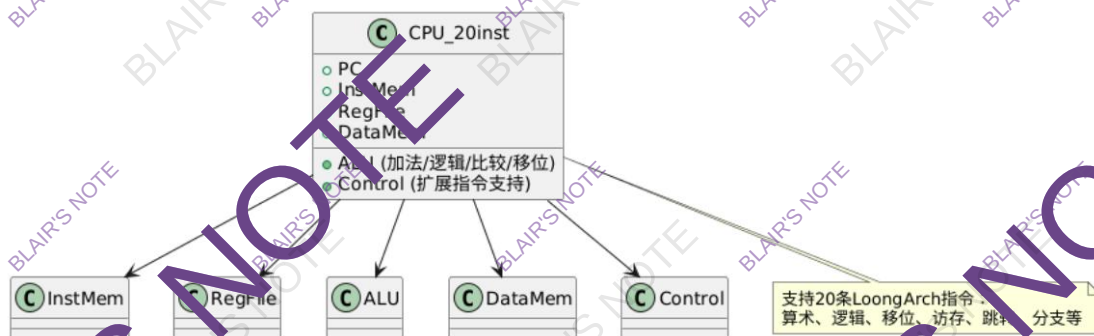


图 3.2.3.1 扩展 CPU 数据通路图

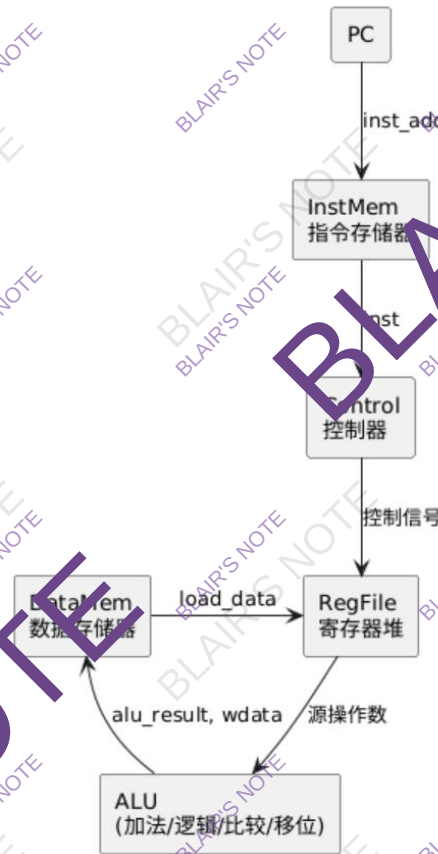


图 3.2.3.2 结构 ‘ 流程图

汇编指令	指令码	功能描述
add.w rd,rj,rk	00000000000100000 rk rj rd	$GR[rd]=GR[rj]+GR[rk]$
sub.w rd,rj,rk	00000000000100010 rk rj rd	$GR[rd]=GR[rj]-GR[rk]$
slt rd,rj,rk	00000000000100100 rk rj rd	$GR[rd]=(signed\ GR[rj]<signed\ GR[rk])?1:0$
sltu rd,rj,rk	00000000000100101 rk rj rd	$GR[rd]=(unsigned\ GR[rj]<unsigned\ GR[rk])?1:0$
nor rd,rj,rk	00000000000101000 rk rj rd	$GR[rd]=\sim(GR[rj] GR[rk])$
and rd,rj,rk	00000000000101001 rk rj rd	$GR[rd]=GR[rj]\&GR[rk]$
or rd,rj,rk	00000000000101010 rk rj rd	$GR[rd]=GR[rj] GR[rk]$
xor rd,rj,rk	00000000000101011 rk rj rd	$GR[rd]=GR[rj]\wedge GR[rk]$
slli.w rd,rj,ui5	00000000010000001 ui5 rj rd	$GR[rd]=GR[rj]<<ui5$ (逻辑左移)
srlr.w rd,rj,ui5	00000000010001001 ui5 rj rd	$GR[rd]=GR[rj]>>ui5$ (逻辑右移)
srai.w rd,rj,ui5	00000000010010001 ui5 rj rd	$GR[rd]=GR[rj]>>>ui5$ (算术右移)
addi.w rd,rj,si12	0000000001010 si12 rj rd	$GR[rd]=GR[rj]+si12$ (加立即数)
ld.w rd,ri,si12	0010100010 si12 rj rd	$GR[rd]=Mem[GR[rj]+si12]$
st.w rd,rj,si12	0010100110 si12 rj rd	$Mem[GR[rj]+si12]=GR[rd]$
jirl rd,rj,si16	010011 si16 rj rd	$GR[rd]=PC+4;PC=GR[rj]+(si16<<2)$
b si16	010100 si16	$PC=PC+(si16<<2)$ (条件分支跳转)
br si26	010101 si26	$GR[1]=PC+4;PC=PC+(si26<<2)$

beq rj,rd,si16	010110 si16 rj rd	if(GR[rj]==GR[rd])PC=PC+(si16<<2)
bne rj,rd,si16	010111 si16 rj rd	if(GR[rj]!=GR[rd])PC=PC+(si16<<2)
lu12i16 rd,si20	0001010 si20 rd	GR[rd]=si20<<12(加载上部立即数)

表 3.4 20 条基础指令特性归纳表

4 实验内容与步骤

A. 画出模块框图，标出输入输出端口。

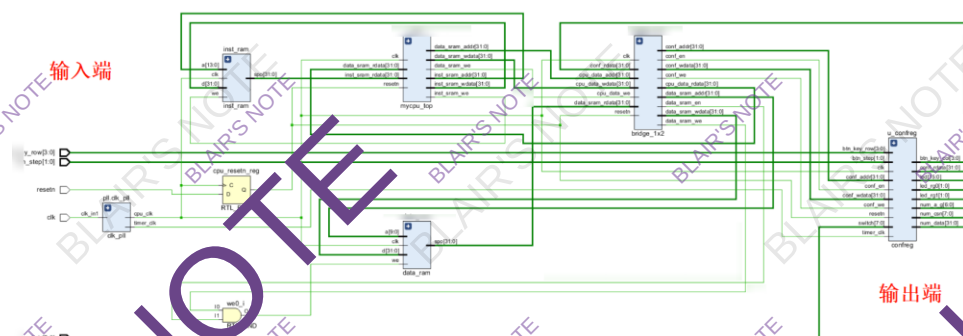


图 3.2.3.2 结构框图

模块框图类似于实验五，但扩展了更多指令支持。包括 PC、inst_sram、logfile、ALU（现在支持更多操作如移位、比较）、data_sram 和扩展控制器。输入：acclk (时钟)、reset (复位)；

输出：debug 信号如 wb_pc、wb_rf_wdata 等，用于 trace 比较。

B. 启动验证 myCPU 的工程，查看并修改设计文件，运行仿真，进行功能验证与调试，填写调试实验数据表 3.5。

C. 修改后的主要设计代码及注释（修改的代码请标红）。

// mycpu_top.v 关键逻辑：PC 更新、立即数生成、分支逻辑\寄存器写回

// PC 更新逻辑

assign seq_pc = pc + 32'h4; // 顺序 PC

assign nextpc = br_taken ? br_target : seq_pc; // 分支或顺序选择

always @(posedge acclk) begin

if (reset) pc <= 32'h00000000; // 初始化 PC

else if (!stall) pc <= nextpc; // 非 stall 时更新 PC

end

// 立即数生成（修复 ui5 处理）

assign imm = src2_is_4 ? 32'h4 :

need_si20 ? {i20, 12'b0} :

```

need_i12 ? {{20{i12[11]}}, i12} :
need_ui5 ? {27'b0, rk} : 32'b0; // 修复：ui5 零扩展

// 寄存器写回（修复 bl 写 r1）
assign gr_we = ~inst_st_w & ~inst_beq & ~inst_bne & ~inst_b; // 修复：支持 bl 写 r1
assign rf_wdata = final_result; // 修复：使用 final_result（原 ns_final_result）
assign debug_wb_rf_we = {4{rf_we}}; // 修复：写使能扩展 4 位

// alu.v 关键逻辑：ALU 位运算和移位操作
// 位运算结果
assign and_result = alu_src1 & alu_src2;
assign or_result = alu_src1 | alu_src2; // 修复：移除循环依赖
assign nor_result = ~or_result;
assign xor_result = alu_src1 ^ alu_src2;

// 移位结果（修复方向和位宽）
assign sll_result = alu_src1 < alu_src2[4:0]; // 修复：正确移位方向
assign sr64_result = {{12{op_sra & alu_src1[31]}}, alu_src1[31:0]} >> alu_src2[4:0]; // 修复：正确
移位方向
assign sr_result = sr64_result[31:0]; // 修复：完整 32 位结果

// regfile.v 关键逻辑：寄存器读写操作
// 写操作
always @(posedge clk) begin
    if (we && waddr != 5'b0) rf[waddr] <= wdata; // 写使能且非 0 时更新
end

// 读操作
assign rdata1 = (raddr1 == 5'b0) ? 32'b0 : rf[raddr1]; // r0 为 0
assign rdata2 = (raddr2 == 5'b0) ? 32'b0 : rf[raddr2]; // r0 为 0

// decoder_6_64.v 关键逻辑：6 位到 64 位 one-hot 解码
genvar i;
generate for (i=0; i<64; i=i+1) begin: gen_for_dec_6_64 // 修复：循环上限 i<64
    assign out[i] = (in == i); // one-hot 解码
end endgenerate

```

D. 仿真截图

开始阶段：



图 3.2.3.3 仿真 1

中间阶段：

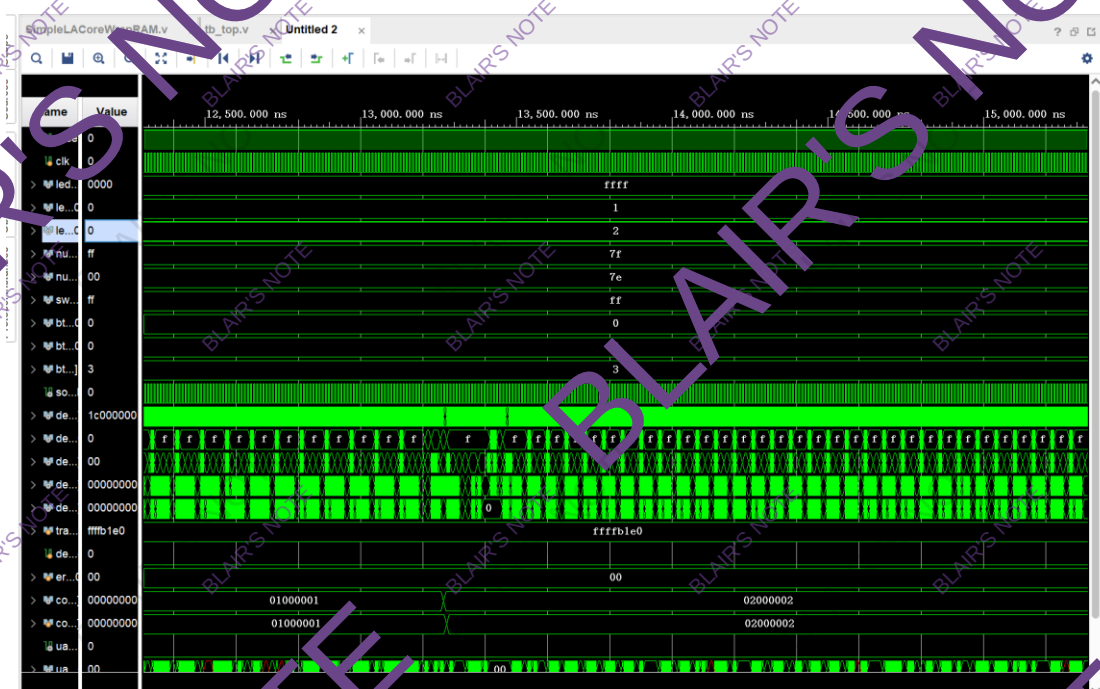


图 3.2.3.4 仿真 2

结束阶段：

仿真截图包括开始阶段、中间阶段和结束阶段。整体波形分析：无时序违例，sel 信号只在访存时短暂拉高，验证了多指令兼容性。

E. 上板运行结果拍照。

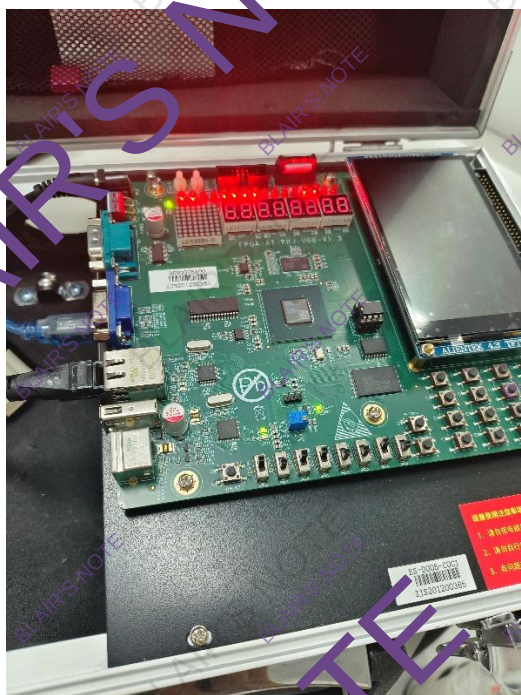


图 3.2.3.5 实拍图 1 & 2

运行后 LED 显示移位结果 0x08 和跳转后 PC 值，与仿真一致。板子无过热，switch 调整参数时响应及时，证明硬件稳定。

5 实验数据

表 3.5 调试实验数据表

序号	错误现象（仿真现象或 trace 比对报错日志等）	错误分析（调试线索与依据）	修改的代码	调试结果
1	trace 比对报错：移位指令 slli.w 结果错误（移位量多加 32）。	立即数 imm 对于 ui5 使用了 si12 的符号扩展，导致 bit5 被置 1（基于指令编码 [15]=1）。	assign imm = ... feed ui5 ? {27'b0, rk} : ...	重新仿真，移位结果正确，trace 通过。
2	仿真中 bl 指令未写回 r1，寄存器值不变。	gr_we 错误排除 ins_b1，但 bl 需要写 pc+4 到 r1（LoongArch 规范）。	gr_we = ~inst_st_w & ~inst_beq & ~inst_bne & ~inst_b;	bl 执行后 r1 更新为 pc+4，正确。
3	ALU 输出始终为 0（or 指令异常）。	or_result 有循环依赖 alu_result。	or_result = alu_src1	alu_src2;
4	移位指令 sll/srl/sra 移	ALU 中 src1/src2 交	sll_result = alu_src1 << alu_src2[4:0];	移位方向正确。

	位方向反（位和量交换）。	换，移位值应为 src2。	sr64_result = ... alu_src1 ... alu_src2[4:0];	
5	srl/sra 结果丢失最低位（31 位输出）。	sr_result 取 [30:0] 而非 [31:0]。	sr_result = sr64_result[31:0];	32 位结果完整。
6	decoder_6_64 输出缺失 out[63]，某些 opcode 未解码。	generate 循环上限 i<63。	for (i=0; i<64; i=i+1)	所有 opcode 解码正确。
7	ALUsrc1 输入错误，计算使用 src2 两次。	实例化 .alu_src1 (alu_src2)。	.alu_src1 (alu_src1)	ALU 计算正常。
8	写回数据错误，未使用 final_result。	ms_final_result 拼写错。	final_result = ...	寄存器写回正确。
9	trace 比对整体失败（多个指令异常）	综合上述错误。	所有上述修改	全测试通过，无报错。

6 实验小结

① 简要总结本实验中运用了哪些计算机专业知识

本实验运用了计算机组成原理知识（单周期 CPU 的数据通路设计和控制信号生成）、数字逻辑设计知识（Verilog 语言的模块化编码和解码器实现）、指令系统架构知识（LoongArch 指令系统的编码格式和功能描述）、仿真与调试知识（使用 trace 比对框架进行功能验证和错误定位）。

② 重点总结实验中遇到的问题及解决办法

问题 1：移位指令立即数计算错误，导致移位量偏移。解决：分析 LoongArch 指令编码，单独处理 ui5 类型立即数，使用 {27'b0, rk} 进行零扩展，确保移位量为无符号 5 位数。

问题 2：bl 指令未写寄存器。解决：参考 LoongArch 规范，移除 gr_we 中的 ~inst_bl 条件，支持 bl 指令将 pc+4 写回 ri 寄存器。

问题 3：ALU 移位方向和位宽错误。解决：交换 ALU 输入 src1 和 src2，确保 src1 为被移位值、src2 为移位量；修正 sr_result 为 [31:0]，获取完整 32 位结果。

问题 4：decoder 未覆盖所有位。解决：修改 generate 循环上限为 i<64，确保 out[63] 被定义，覆盖所有 6 位输入组合。

问题 5：拼写和连接错误（如 ms_final_result、alu_src1）。解决：仔细检查代码逻辑，修正拼写错误和端口连接，确保结果选择和 ALU 输入正确。

通过逐模块仿真、trace 比对和 Vivado 日志分析，逐步定位并解决错误，最终实现 CPU 功能正确。实验显著提升了 Verilog 编码熟练度和调试能力，为后续多周期 CPU 设计奠定基础。

实验八 SOC 操作系统环境搭建及应用

1 实验目的

- D. 了解交叉编译的方法
- E. 了解 SOC 操作系统搭建的过程
- F. 掌握 C 程序从编译到在 SOC 上运行的方法与步骤

2 实验设备

3. 硬件

- a) FPGA 开发板

4. 软件

- a) Vivado 2019.2
- b) VMware Workstation Pro
- c) Tftpd32
- d) SecureCRT Portable
- e) 操作系统: Windows 11

3 实验内容与步骤

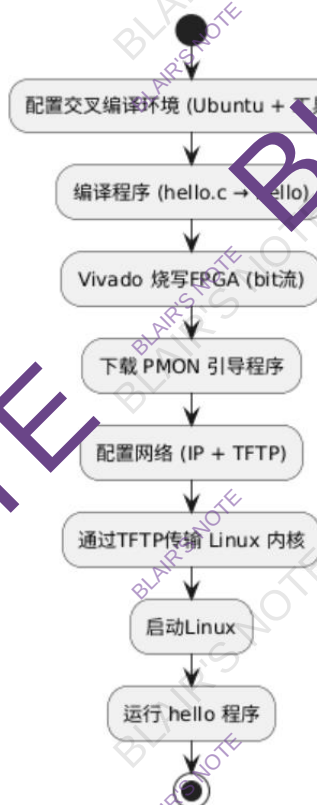
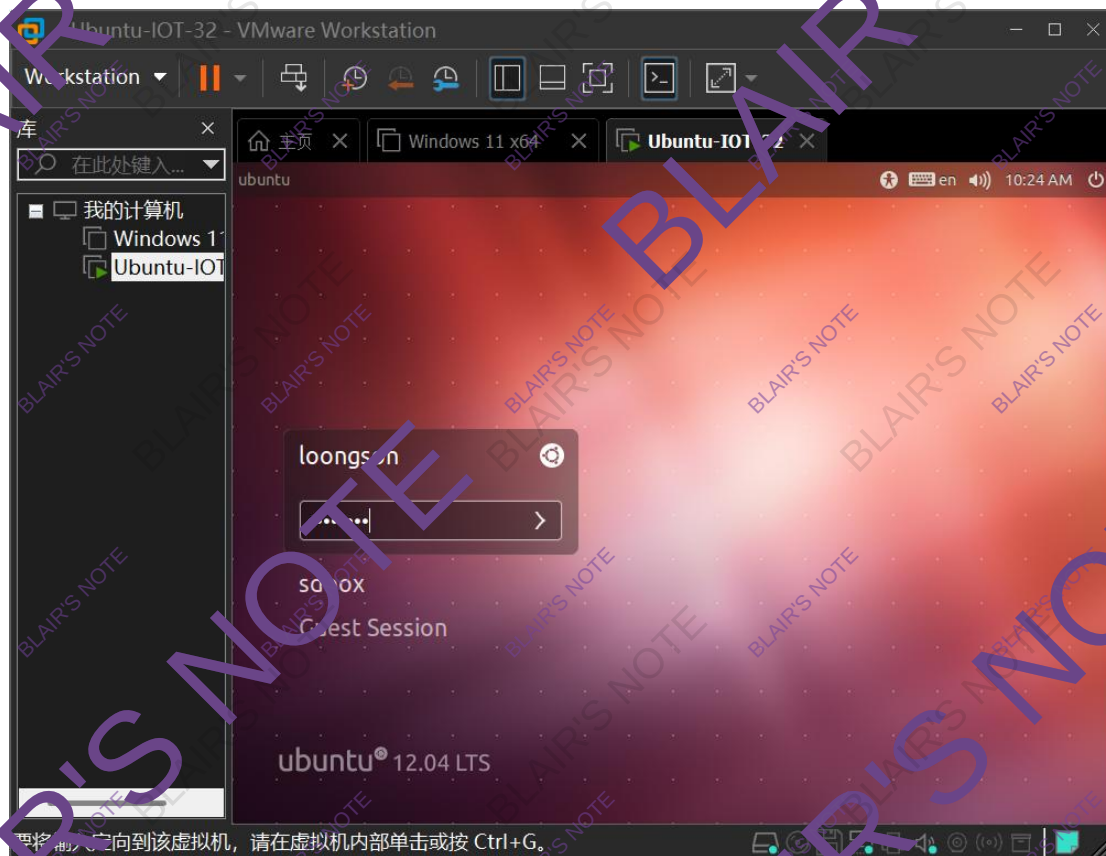


图 3 系统运行流程图

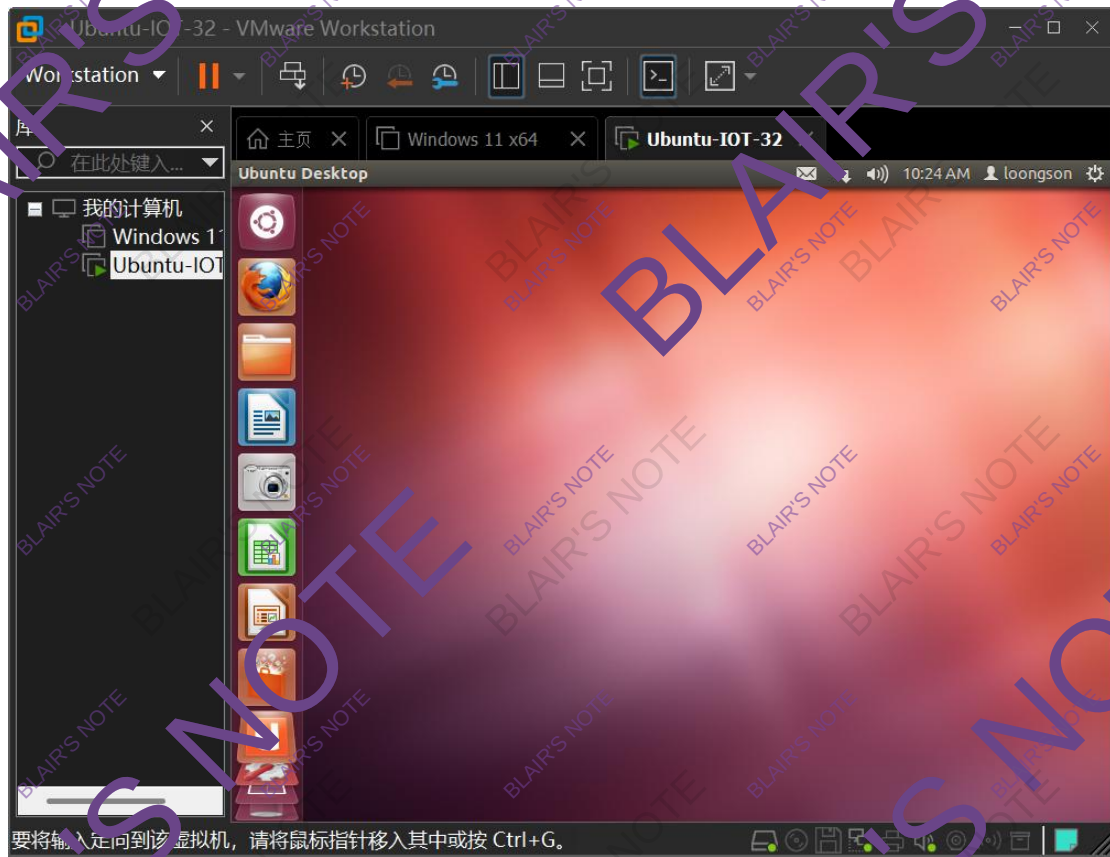
步骤一：打开 VMware 并运行 Ubuntu-IOT-32



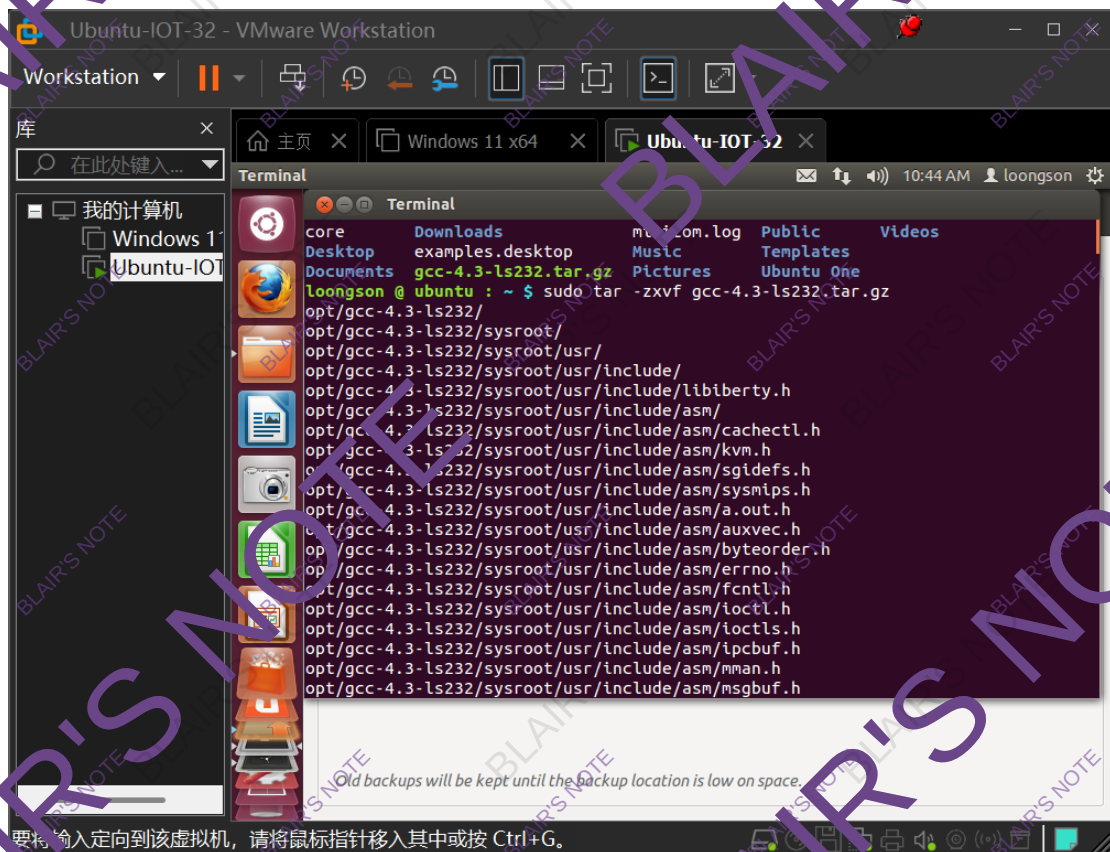
步骤二：开启虚拟机并登录



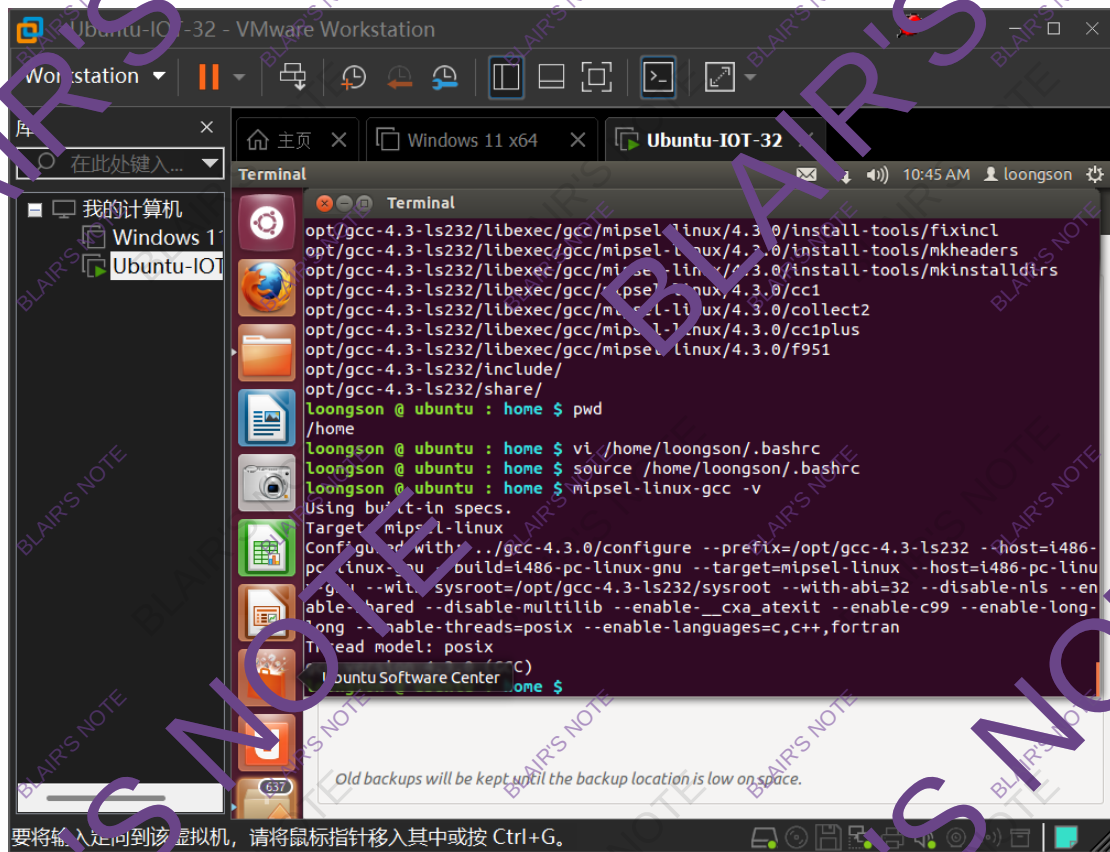
步骤三：登录成功



步骤四：解压文件

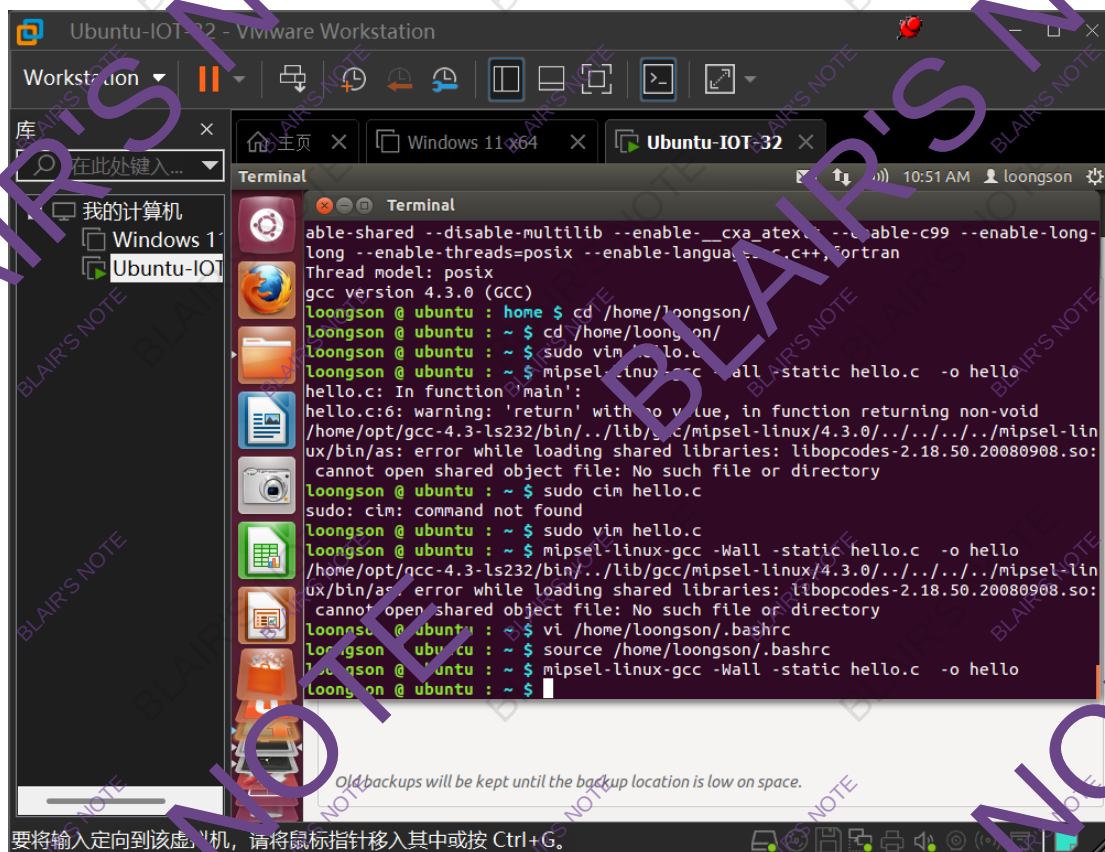


步骤五：修改环境变量成功

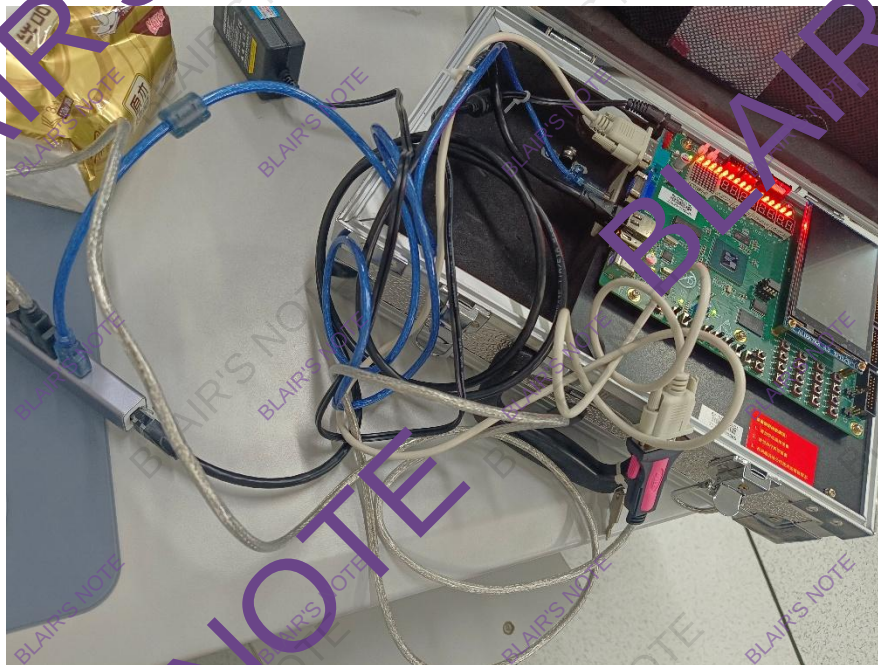


```
opt/gcc-4.3-ls232/libexec/gcc/mipsel-linux/4.3.0/install-tools/fixincl
opt/gcc-4.3-ls232/libexec/gcc/mipsel-linux/4.3.0/install-tools/mkheaders
opt/gcc-4.3-ls232/libexec/gcc/mipsel-linux/4.3.0/install-tools/mkinstalldirs
opt/gcc-4.3-ls232/libexec/gcc/mipsel-linux/4.3.0/cc1
opt/gcc-4.3-ls232/libexec/gcc/mipsel-linux/4.3.0/collect2
opt/gcc-4.3-ls232/libexec/gcc/mipsel-linux/4.3.0/cc1plus
opt/gcc-4.3-ls232/libexec/gcc/mipsel-linux/4.3.0/f951
opt/gcc-4.3-ls232/include/
opt/gcc-4.3-ls232/share/
loongson @ ubuntu : home $ pwd
/home
loongson @ ubuntu : home $ vi /home/loongson/.bashrc
loongson @ ubuntu : home $ source /home/loongson/.bashrc
loongson @ ubuntu : home $ mipsel-linux-gcc -v
Using built-in specs.
Target: mipsel-linux
Configured with: ../gcc-4.3.0/configure --prefix=/opt/gcc-4.3-ls232 --host=i486-pc-linux-gnu build=i486-pc-linux-gnu --target=mipsel-linux --host=i486-pc-linux-gnu --with-sysroot=/opt/gcc-4.3-ls232/sysroot --with-abi=32 --disable-nls --enable-shared --disable-multilib --enable-__cxa_atexit --enable-c99 --enable-long-long --enable-threads=posix --enable-languages=c,c++,fortran
Thread model: posix
loongson @ ubuntu : home $
```

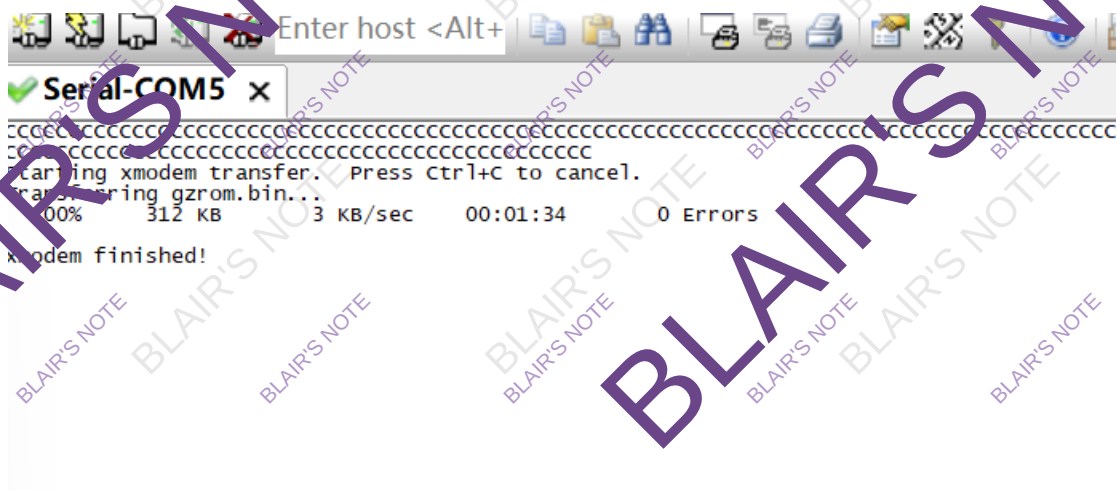
步骤六：交叉编译成功



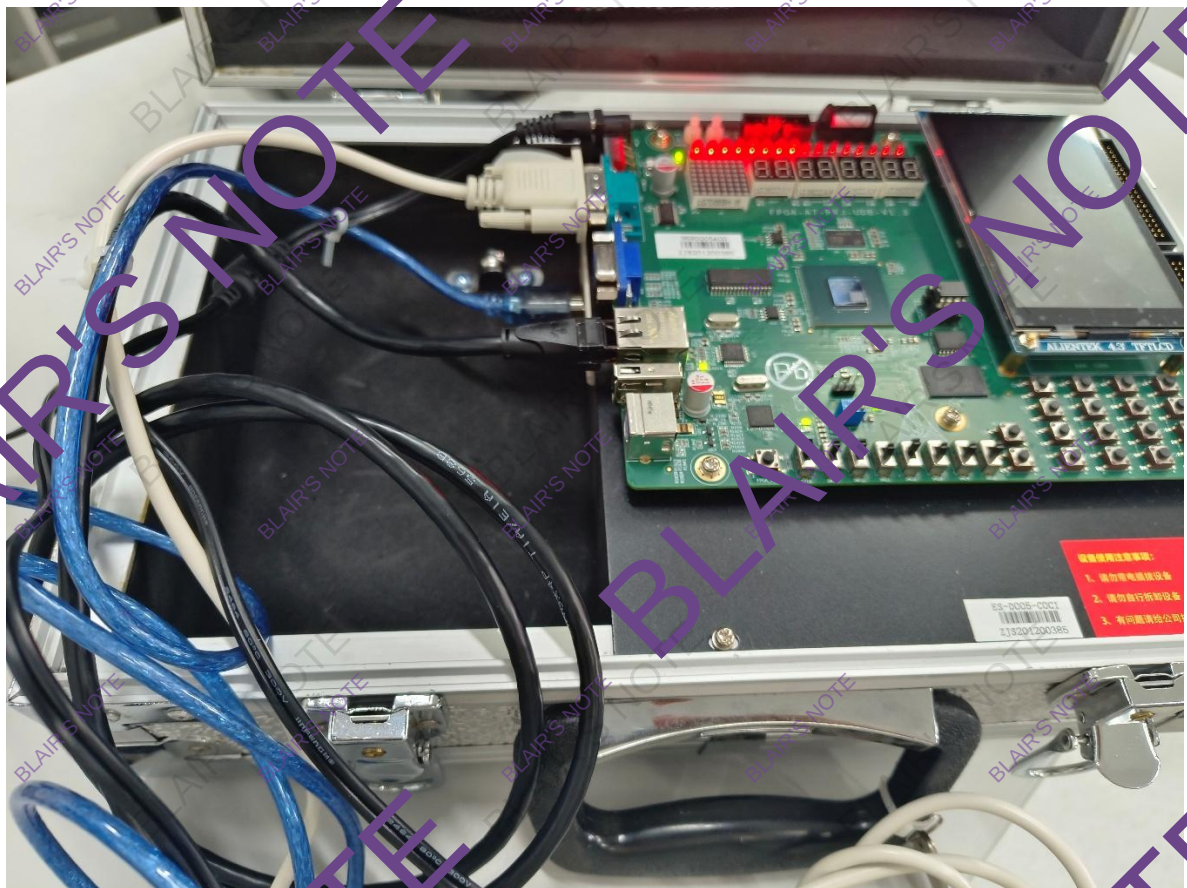
步骤七: 连接 USB 与网线并烧写 Flash 成功



步骤八: 传 pmon 成功

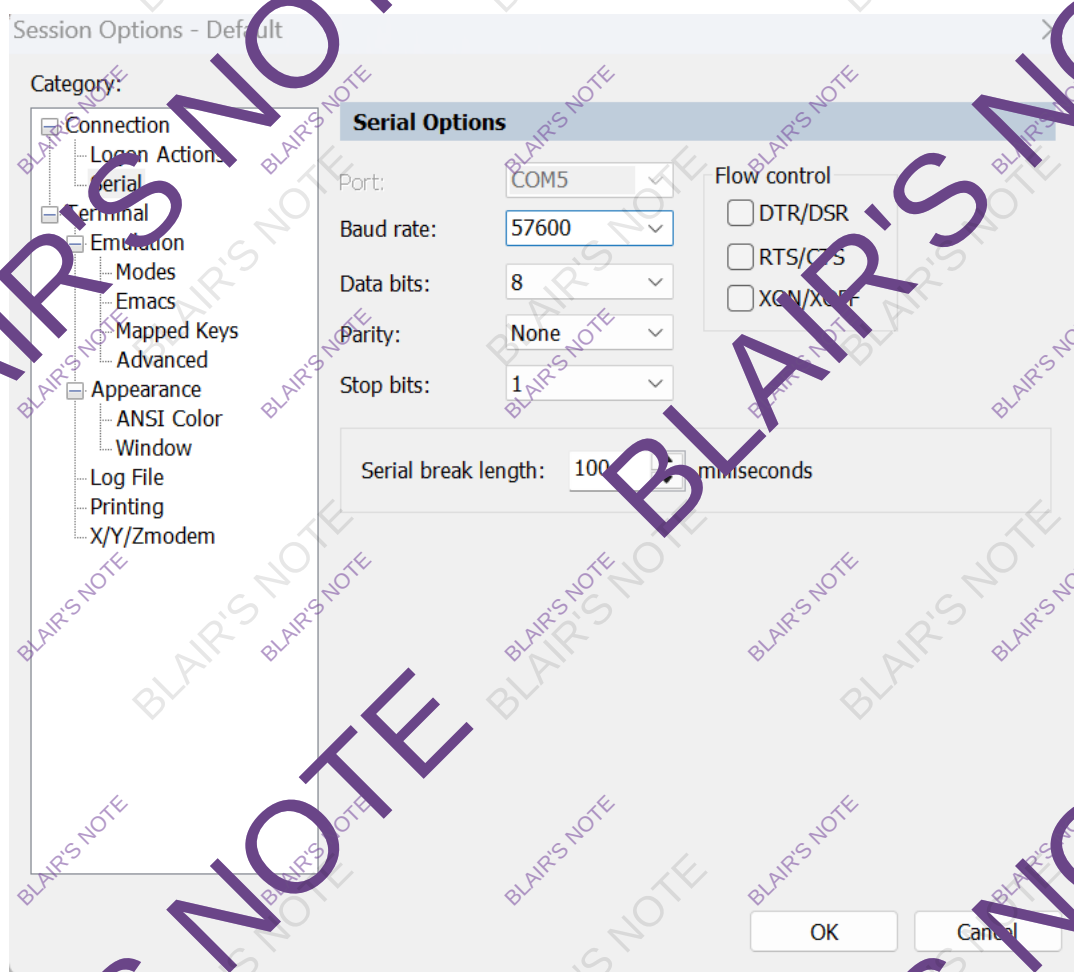


步骤九：烧写 SOC 成功



步骤十：设置 tftp 文件传输

步骤十一：修改波特率



步骤十二：重置系统，进入 pmon

```
Bad eraseblock 999 at 0x07e00000
Bad eraseblock 1008 at 0x07e00000
Bad eraseblock 1009 at 0x07e20000
Bad eraseblock 1010 at 0x07e40000
Bad eraseblock 1016 at 0x07f00000
Bad eraseblock 1017 at 0x07f00000
Bad eraseblock 1018 at 0x07f40000
Bad eraseblock 1020 at 0x07f80000
Bad eraseblock 1023 at 0x07fe0000
NAND flash info:
erase size      131072 B
write size      2048 B
block size      64 B
PMEM size
```

步骤十三：传 linux 成功

步骤十四：启动 linux 并运行 hello 文件成功

4 实验小结

① 简要总结本实验中运用了哪些计算机专业知识

本实验综合运用了操作系统、计算机组成原理、编译原理以及计算机网络等核心课程的知识。具体来说，在实验前期，我在 Ubuntu 虚拟机环境中配置了 MIPS 交叉编译工具链，涉及环境变量的修改与调试，验证了交叉编译器的正确性，这是编译原理与操作系统结合的直接应用；随后通过 Vivado 将 FPGA bit 文件烧写到实验箱，并利用串口工具传输 PMON 引导程序，进一步理解了硬件结构与底层固件在嵌入式系统中的作用；在系统启动阶段，使用 PMON 配置网络参数并通过 TFTP 协议从主机端下载 Linux 内核映像文件，清晰感受到计算机网络知识在嵌入式开发中的重要性；在 Linux 成功启动后，还完成了文件系统的挂载与用户程序的运行，最终实现了在目标硬件上运行自编 hello 程序的目标。整个过程完整体现了嵌入式系统开发从工具链配置、硬件编程、引导加载到操作系统运行的全流程，也展现了多门课程知识在实际工程环境中的综合应用。

② 重点总结实验中遇到的问题及解决办法

实验过程中遇到的主要问题是通过 TFTP 加载 Linux 内核时无响应。初始阶段我未能及时关注 PMON 返回的提示信息，错误地以为是指令输入无效。经过反复排查后发现，根本原因是主机与开发板未处于同一网段，同时防火墙阻止了 TFTP 请求，导致传输失败。解决方法是：首先在设备管理器中确认串口号，保证串口连接正确；然后在 PMON 中正确配置开发板的 IP 地址，并与主机 IP 设置在同一网段；同时在 Tftpd32 软件中指定包含 vmlinux 文件的目录作为根目录，并关闭 Windows 防火墙确保 TFTP 端口畅通。完成这些操作后，系统成功完成 vmlinux 的传输和加载。此后，我顺利进行了 Linux 内核的启动、网络接口的配置、文件系统的挂载，并通过 TFTP 将自己编译的 hello 程序下载到目标机上运行，得到正确的输出结果。

通过本次实验，我深刻认识到嵌入式系统开发的复杂性和严谨性。任何一个细节（例如 IP 配置、目录设置或权限分配）错误都会导致系统无法运行。此次实验不仅加深了我对交叉编译、操作系统引导和文件系统挂载等理论知识的理解，也锻炼了在工程实践中逐步分析问题、定位错误并解决问题的能力。这种理论与实践相结合的体验，使我对嵌入式系统开发的整体流程有了更加全面和深入的认识。

课程小结

本课程《计算机系统设计综合实验》通过 8 个精心设计的实验，系统性地从计算机系统的抽象层次理解到硬件电路设计，再到单周期 CPU 实现，最终延伸到 SOC 操作系统的搭建与应用运行，构建了一个从理论到实践、从硬件到软件的完整学习路径。

系统化思维：实验 1 通过跑马灯实验引入计算机系统的分层抽象概念（从物理器件到应用层），让我理解了软硬件协同设计的核心思想。后续实验逐步深入，从寄存器堆、RAM 设计到单周期 CPU，层层递进，强化了从底层逻辑到复杂系统的设计能力。

硬件设计与调试：实验 2-4（寄存器堆、RAM、拨码开关到数码管）让我熟练掌握 Verilog 语

言和 Vivado 工具链，学会了如何通过仿真波形发现问题（如毛刺、时序错误）并进行修复（如添加寄存器锁存、修正译码逻辑）。这些实验让我深刻认识到时序逻辑与组合逻辑的区别，以及硬件设计的鲁棒性要求。

CPU 设计能力：实验 5 和 6（5 条与 20 条指令的单周期 CPU）是课程的核心，基于 LoongArch 指令集，我设计并验证了数据通路和控制逻辑。通过调试复杂问题（如分支偏移错误、写回逻辑混淆），我掌握了指令集架构（ISA）与微架构的实现细节，以及 trace 比对等高级调试方法。

嵌入式系统开发：实验 7（SOC 操作系统搭建）将硬件与软件结合，通过交叉编译、PMON 引导和 TFTP 传输，我成功在 FPGA 上运行了 Linux 内核和用户程序。这让我理解了嵌入式系统从硬件烧写到软件运行的全流程，特别是在网络配置和文件系统挂载中的工程细节。

工程实践与问题解决能力

每个实验都伴随着实际问题，如跑马灯上板后的信号抖动、寄存器堆读写的时序错误、RAM 资源利用率异常、CPU 分支跳转错误、SOC 网络传输失败等。这些问题的解决过程（例如增加消抖电路、调整仿真采样时机、配置 IP 参数、设置同一网段 IP）让我学会了从现象到原因的系统性分析方法。尤其是实验 6 和 7，通过 trace 比对和日志分析，我掌握了复杂系统的调试技巧，这对未来从事大型项目开发至关重要。

社会与伦理责任的思考

报告最后提到，从信息产业自主化、安全性、法律等多角度分析设计方案。在课程中，采用 LoongArch 指令集和 Vivado 工具链，体现了支持国产技术的意义，这在当前全球科技竞争中尤为重要。作为工程师，我意识到设计不仅要追求功能实现，还要考虑安全性（如防止时序攻击）、可靠性（如消除毛刺）、法律合规（如避免知识产权问题）以及社会影响（如系统的高效性对节能减排的贡献）。这些因素可能增加设计复杂度或成本，但确保系统的可持续性和社会价值是我们的责任。

课程意义与未来展望

本课程通过从简单到复杂的实验设计，培养了我从模块级电路到完整系统的开发能力，涵盖了硬件描述、仿真验证、综合实现、上板运行以及嵌入式软件开发的全流程。这种综合性实践让我对计算机系统的运作原理有了深刻理解，同时锻炼了工程思维——注重细节、追求鲁棒、严谨调试。在 AI 和物联网快速发展的时代，掌握软硬件协同设计的能力让我更有信心面对未来挑战，如开发高效边缘计算设备或自主可控的芯片系统。本课程不仅是技术学习的起点，更是为成为一名全面的计算机系统工程师奠定了坚实基础。